

Don't Get Your Kroneckers in a Twist: Gaussian Processes on High-Dimensional Incomplete Grids

Mads Greisen Højlund
Department of Chemistry
Aarhus University
madsgh@chem.au.dk

August Smart Lykke-Møller
Department of Chemistry
Aarhus University
alm@chem.au.dk

Henry Moss
School of Mathematical Sciences
Lancaster University
henry.moss@lancaster.ac.uk

Ove Christiansen
Department of Chemistry
Aarhus University
ove@chem.au.dk

Abstract

We introduce CUTS-GPR, a new method for performing numerically exact Gaussian process regression (GPR) in high-dimensional settings. The key component of CUTS-GPR is an extremely fast kernel matrix-vector product, which exhibits near-linear or even linear scaling with the amount of training data, N , and low-order polynomial scaling with dimensionality, D . This is obtained by combining an *additive kernel* with an *incomplete grid* and exploiting the resulting structure of the kernel matrix. We demonstrate the scalability of the matrix-vector product by running benchmarks with billions of data points and thousands of dimensions. Full GPR calculations, including hyperparameter optimization, are completed in a matter of hours for $N = 447\,265$ and $D = 24$. We demonstrate that our CUTS-GPR enables Bayesian modeling of high-dimensional potential energy surfaces – a longstanding challenge in computational chemistry.

1 Introduction

Gaussian process regression (GPR) is a powerful nonparametric, probabilistic machine learning method that enables predictions with quantified uncertainty and modeling of complex data. For many applications in science and engineering – where accuracy, interpretability, and reliable uncertainty estimates are essential – GPR is highly attractive. It is particularly well suited to settings where data is expensive and error control is critical. However, the high computational cost of GPR remains a major limitation. For N training points, conventional GPR implementations require $\mathcal{O}(N^3)$ operations and $\mathcal{O}(N^2)$ storage, which becomes prohibitive for large datasets. The computational cost of GPR becomes particularly severe in high-dimensional spaces. As the dimensionality, D , increases, the data and computation requirements quickly compromise practicality – this is the *curse of dimensionality*. GPR could potentially see much broader adoption if (near-)linear scaling with N and low scaling with D were achievable at the same time.

Near-linear scaling with N can in fact be obtained if the training data is located on a complete Cartesian product grid [1–5], but N in turn scales exponentially with D and the curse of dimensionality remains. In many scientific applications (such as molecular simulations and materials modeling) D can be large, so existing grid-based GPR methods are not sufficient. With CUTS-GPR, we show that grid structure can be exploited to drastically lower computational cost even though the grid is not complete. Incomplete grids are useful for sampling high-dimensional functions such as potential energy surfaces (PESs), which in turn play a central role in understanding and predicting chemical

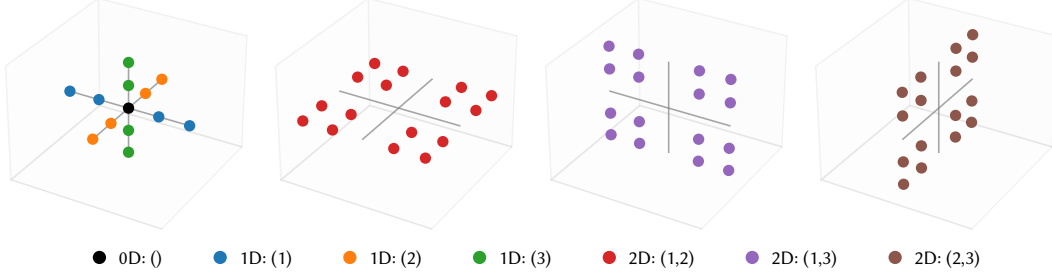


Figure 1: An incomplete grid in three dimensions with a cut level of $\alpha = 2$. The grid includes the reference point (0D), 1D cuts and 2D cuts. A subgrid is defined by its *mode combination* (MC), which is simply the list of dimensions or modes that are displaced from the reference. The red 2D cut labelled by (1, 2) thus contains points displaced along dimensions 1 and 2. An incomplete grid is defined by its *mode combination range* (MCR), which is simply the list of MCs contained.

processes. CUTS-GPR thus paves the way for truly high-dimensional applications in science and elsewhere. In particular, we make the following contributions:

- (i) We introduce a class of structured, incomplete grids suitable for high-dimensional applications. The grid includes a reference point and a set of low-dimensional subgrids or cuts (see Figure 1). In the simplest case, we include all subgrids of order $0, 1, 2, \dots, \alpha$, in which case the amount of training data scales as $\mathcal{O}(D^\alpha)$.
- (ii) Combining this type of grid with an additive kernel, we derive and implement a kernel matrix-vector product (MVP) with computational complexity only $\mathcal{O}(n\alpha N)$ (assuming n grid points per dimension). Focusing on D , the complexity is only $\mathcal{O}(D^\alpha)$. The key to achieving this remarkable performance is the careful use of structure in the kernel matrix. No approximation is involved in the kernel MVP.
- (iii) The availability of a fast, scalable MVP in turn allows predictions and hyperparameter optimization using iterative numerical techniques. This enables high-dimensional GPR with (near-)linear scaling with N and polynomial scaling with D , which is a major breakthrough for many applications of GPR.

1.1 Related work

A substantial literature has sought to address the $\mathcal{O}(N^3)$ computational cost of conventional GPR. Sparse inducing-point methods, such as FITC [6], yield approximate posteriors with improved scalability [6–8]. In a complementary direction, structured GP models leverage Toeplitz or Kronecker structure, enabling exact inference at much reduced complexity [1–5]. Kronecker-based methods are suitable for multidimensional data, but their reliance on complete grids leads to exponential scaling with dimensionality. The three strategies may also be combined as in KISS-GP [9] or SKIP [10], the latter of which improves scalability with D as well as N . Despite these advances, large-scale and exact GPR for truly high-dimensional problems remains unresolved.

2 Background

2.1 Gaussian process regression

The training data is a set of D -dimensional inputs $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^N$ along with the corresponding noisy outputs, which are collected in a vector $\mathbf{y} \in \mathbb{R}^N$. The task is now to predict the unknown outputs at a set of test points $\mathcal{X}_* = \{\mathbf{x}_i^*\}_{i=1}^{N_*}$, and for this purpose we need the training–training, training–test and test–test covariance matrices, which are defined in terms of the kernel function, k :

$$\mathbf{K} = k(\mathcal{X}, \mathcal{X}), \quad \mathbf{K}_* = k(\mathcal{X}, \mathcal{X}_*), \quad \mathbf{K}_{**} = k(\mathcal{X}_*, \mathcal{X}_*). \quad (1)$$

We also introduce the noisy training–training covariance matrix, $\mathbf{C} = \mathbf{K} + \sigma^2 \mathbf{I}$, and the weights, $\boldsymbol{\alpha} = \mathbf{C}^{-1} \mathbf{y}$. With these definitions in place, we can compute the predictive mean and covariance as

$$\boldsymbol{\mu} = \mathbf{K}_*^\top \mathbf{C}^{-1} \mathbf{y} = \mathbf{K}_*^\top \boldsymbol{\alpha}, \quad \boldsymbol{\Sigma} = \mathbf{K}_{**} - \mathbf{K}_*^\top \mathbf{C}^{-1} \mathbf{K}_*. \quad (2)$$

Any hyperparameter, θ , can be determined by maximizing the marginal log-likelihood (MLL),

$$\mathcal{L} = -\frac{1}{2} (\mathbf{y}^\top \boldsymbol{\alpha} + \log |\mathbf{C}| + N \log 2\pi). \quad (3)$$

For optimization using a gradient-based algorithm, we also require the derivatives:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{1}{2} \boldsymbol{\alpha}^\top \frac{\partial \mathbf{C}}{\partial \theta} \boldsymbol{\alpha} - \frac{1}{2} \text{tr} \left(\mathbf{C}^{-1} \frac{\partial \mathbf{C}}{\partial \theta} \right). \quad (4)$$

GPR thus requires the following ingredients:

- (i) Linear solves with $\mathbf{C}$, for example $\boldsymbol{\alpha} = \mathbf{C}^{-1} \mathbf{y}$;
- (ii) The log-determinant, $\log |\mathbf{C}| = \text{tr} \log(\mathbf{C})$;
- (iii) The trace term, $\text{tr}(\mathbf{C}^{-1} \partial \mathbf{C} / \partial \theta)$;
- (iv) The quadratic term, $\boldsymbol{\alpha}^\top (\partial \mathbf{C} / \partial \theta) \boldsymbol{\alpha}$.

Conventional implementations use the Cholesky decomposition of $\mathbf{C}$, which takes $\mathcal{O}(N^3)$ time to compute and $\mathcal{O}(N^2)$ space to store. Having obtained the decomposition, one can compute the quantities listed above directly in at most $\mathcal{O}(N^2)$ time. Often $\mathbf{K}$ (and thus $\mathbf{C}$) allow fast MVPs, which is sufficient for numerically exact predictions and accurate stochastic estimates of the MLL and the MLL gradient [11, 12]. Appendix A recalls the main components of MVP-based GPR, including the use of preconditioning. In the following, we focus on developing a fast, low-scaling kernel MVP.

2.2 Complete grids

Throughout the paper, we are concerned with D -dimensional inputs $\mathbf{x}$ that are located on a grid. We will start by defining a *complete* grid, by which we mean a Cartesian product grid like

$$\mathcal{X} = \mathcal{X}^{(1)} \times \dots \times \mathcal{X}^{(D)} \subset \mathbb{R}^D. \quad (5)$$

Each $\mathcal{X}^{(d)}$ is a 1D grid (not necessarily regular) of size n_d , and the full grid has size $N = |\mathcal{X}| = n_1 n_2 \dots n_D$. It is natural in this setting to index each 1D grid by an index set $\mathcal{I}^{(d)} = \{0, 1, \dots, n_d - 1\}$ and each multidimensional grid point by a multi-index $\mathbf{i} = (i_1, \dots, i_D) \in \mathcal{I} = \mathcal{I}^{(1)} \times \dots \times \mathcal{I}^{(D)}$. It is well known that the combination of a complete grid and a separable kernel,

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = \prod_{m=1}^D k^{(m)}(x^{(m)}, \tilde{x}^{(m)}), \quad (6)$$

enables GPR with near-linear complexity [1–4]. This is due to the fact that the kernel matrix obtained by evaluating (6) over a complete grid is a Kronecker product. Specifically, one finds that

$$\mathbf{K} = \mathbf{K}^{(1)} \otimes \mathbf{K}^{(2)} \otimes \dots \otimes \mathbf{K}^{(D)}, \quad \mathbf{K}^{(m)} \in \mathbb{R}^{n_m \times n_m}, \quad (7)$$

where the matrices $\mathbf{K}^{(m)}$ are called base matrices. The key properties of Kronecker products are outlined in Appendix B, along with some implications for low-scaling GPR. One can, for instance, compute inverse kernel MVPs with a computational cost of only $\mathcal{O}((n_1 + \dots + n_D)N) = \mathcal{O}(nDN)$, assuming 1D grids of identical size n . In spite of the attractive computational complexity, there are two major downsides to this approach: (1) The separable kernel is often a poor choice in high dimension; and (2) the N required to preserve modelling accuracy scales exponentially with the number of dimensions, thus limiting applications to $D < 10$ or so (depending on n).

2.3 Additive kernels

A more attractive kernel format for high-dimensional applications is the *additive* kernel [13–15], which is typically defined by a maximum interaction order, ω :

$$k = \sigma_0^2 + \sigma_1^2 \sum_m k^{(m)} + \sigma_2^2 \sum_{m < m'} k^{(m)} k^{(m')} + \dots + \sigma_\omega^2 \sum_{m_1 < m_2 < \dots < m_\omega} k^{(m_1)} k^{(m_2)} \dots k^{(m_\omega)}. \quad (8)$$

The hyperparameters σ_k^2 are called order variances and control the relative importance of different interaction orders. In addition, the 1D kernels (base kernels) may each depend on one or more hyperparameters. For definiteness, we will assume that the 1D kernels each depend on a *length scale*,

$\ell^{(m)}$. If $\omega = D$, the kernel is *saturated* and includes all possible interactions. Such a kernel is highly descriptive, but it is usually not possible or necessary to include high-order terms, as shown by Lu et al. [15] who introduced the orthogonal additive kernel (OAK). Compared to a generic additive kernel, the OAK approach includes an extra step where the 1D kernels are orthogonalized or centered [14], which ensures an identifiable model [15].

Ishida et al. [5] introduce a more general and flexible class of additive kernels, which we write as

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = \sum_{\mathbf{m} \in \mathcal{M}_{\text{kernel}}} \sigma_{|\mathbf{m}|}^2 \prod_{m \in \mathbf{m}} k^{(m)}(x^{(m)}, \tilde{x}^{(m)}) \quad (9)$$

where $\mathcal{M}_{\text{kernel}}$ lists the interaction terms included in the kernel. In addition, they argue that $\mathcal{M}_{\text{kernel}}$ should be *hierarchical*, but not necessarily of the simple form implied by (8). Comparing with Section 2.2, it is clear that (9) evaluates to a sum of Kronecker products when using a complete grid, but it is not at all obvious that this kind of structure can be exploited. However, Ishida et al. [5] prove the surprising fact that (14) can be diagonalized in closed form (assuming centered base kernels). Their approach allows GPR with near-linear scaling and a flexible additive kernel for complete-grid datasets. However, the exponential scaling with D remains as a serious limitation.

3 Incomplete grids

To address the curse of dimensionality and motivated by our target applications in computational chemistry, we need a version of grid-based GPR that allows sampling only a subset of the full grid, referred to as an *incomplete* grid. We consider $\hat{N}$ inputs located on an incomplete grid $\hat{\mathcal{X}} \subseteq \mathcal{X}$ indexed by the multi-indices $\hat{\mathcal{I}} \subseteq \mathcal{I}$. We will start by introducing a specific type of incomplete grid and later describe how it relates to a more general class of grids. The incomplete grid used in this paper is defined with respect to a *reference point* that we label by the zero multi-index, $\mathbf{i} = (0, 0, \dots, 0)$. The reference point can be chosen freely according to the problem at hand. In addition to the reference, the grid includes a set of low-dimensional *subgrids* or *cuts*.

Example 3.1. Consider an incomplete grid including the reference, all 1D subgrids and all 2D subgrids. With $D = 3$ this construction includes the points corresponding to the multi-indices

$$\begin{aligned} 0D: & (0, 0, 0) \\ 1D: & (a, 0, 0), (0, b, 0), (0, 0, c) \\ 2D: & (a, b, 0), (a, 0, c), (0, b, c) \end{aligned}$$

for all $0 < a < n_1$, $0 < b < n_2$ and $0 < c < n_3$ (see also Figure 1).

Index sets like Example 3.1 occur frequently in the tensor literature [see e.g. 16] and in vibrational structure theory [17]. In both of these fields, the pertinent dimensions are called *modes*, which is a terminology that we will use in the following. Given a set of 1D grids, a subgrid is uniquely defined by its mode combination (MC), which is simply the list of modes with non-zero indices. An incomplete grid is in turn defined by its mode combination range (MCR), which is simply the list of MCs. Example 3.1 thus corresponds to the MCR

$$\mathcal{M}_{\text{grid}} = \{(), (1), (2), (3), (1, 2), (1, 3), (2, 3)\}. \quad (11)$$

A formal definition of subgrids and incomplete grids is given in Appendix C. In the following, we exclusively consider incomplete grids $\hat{\mathcal{I}}$ whose MCR is *hierarchical* or closed under taking subsets (CUTS):

Definition 3.2. The set $\mathcal{M}$ is closed under taking subsets (CUTS) if $\mathbf{m}' \subset \mathbf{m} \in \mathcal{M}$ implies $\mathbf{m}' \in \mathcal{M}$.

The MCR framework is flexible and can be used to put more emphasis on certain dimensions and less on others. However, it is often useful to define a *simple* MCR that includes all cuts of order $0, 1, 2, \dots, \alpha$. This construction is obviously CUTS and allows definite computational complexities to be derived and stated concisely. Assuming for simplicity that $n_1 = \dots = n_d = n$, we find that an incomplete grid with maximum cut level α has size

$$\hat{N} = \sum_{k=0}^{\alpha} \binom{D}{k} (n-1)^k \sim \mathcal{O}\left(\binom{D}{\alpha} (n-1)^{\alpha}\right) \sim \mathcal{O}\left(\frac{D^{\alpha} n^{\alpha}}{\alpha!}\right). \quad (12)$$

Importantly, we see that $\hat{N}$ grows polynomially with D , in contrast to the exponential growth of the complete grid. Thus, with $\alpha < D$, this data format allows us to mitigate the curse of dimensionality. A cut-based grid with an MCR that it CUTS is just one example of a grid that is closed under decrements (CUD):

Definition 3.3. A set of multi-indices $\hat{\mathcal{I}}$ is closed under decrements (CUD) if

$$(i_1, \dots, i_d, \dots, i_D) \in \hat{\mathcal{I}}, i_d > 0 \Rightarrow (i_1, \dots, i_d - 1, \dots, i_D) \in \hat{\mathcal{I}}.$$

$\hat{\mathcal{I}}$ is also said to be downward closed.

It turns out that being CUD is the crucial property that enables a fast MVP. Moreover, the notion of CUD can be generalized compared to Definition 3.3 [18] (see also Appendix D), which opens the possibility of scalable GPR with other types of incomplete grids.

4 Combining incomplete grids and additive kernels

We now combine the additive kernel with the incomplete grid from Section 3. To that end, we introduce the following convenient notation:

Definition 4.1. Consider the square matrices $\mathbf{A}^{(m)} \in \mathbb{R}^{n_m \times n_m}$ for $m = 1, \dots, D$ and let $N = n_1 n_2 \dots n_D$.

(i) We define the Kronecker product

$$\mathbf{A} = \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(D)} \in \mathbb{R}^{N \times N}$$

via $A_{\mathbf{i}\mathbf{j}} = A_{i_1 j_1}^{(1)} A_{i_2 j_2}^{(2)} \dots A_{i_D j_D}^{(D)}$ for the multi-indices $\mathbf{i} = (i_1, i_2, \dots, i_D)$ and $\mathbf{j} = (j_1, j_2, \dots, j_D)$.

(ii) A one-mode bracket matrix is defined as

$$\mathbf{A}^{[m]} = \mathbf{I}^{(1)} \otimes \dots \otimes \mathbf{A}^{(m)} \otimes \dots \otimes \mathbf{I}^{(D)}$$

with identity matrices in all positions except one.

(iii) A many-mode bracket matrix is defined as

$$\mathbf{A}^{[\mathbf{m}]} = \prod_{m \in \mathbf{m}} \mathbf{A}^{[m]} = \bigotimes_{m=1}^D \mathbf{B}^{(m)}, \quad \mathbf{B}^{(m)} = \begin{cases} \mathbf{A}^{(m)} & \text{if } m \in \mathbf{m}, \\ \mathbf{I}^{(m)} & \text{if } m \notin \mathbf{m}. \end{cases} \quad (13)$$

Introducing the all-ones matrices, $\mathbf{J}^{(m)} \in \mathbb{R}^{n_m \times n_m}$, the complete-grid additive kernel matrix can be written as

$$\mathbf{K} = \sum_{\mathbf{m} \in \mathcal{M}_{\text{kernel}}} \sigma_{|\mathbf{m}|}^2 \mathbf{K}^{[\mathbf{m}]} \mathbf{J}^{[\neg \mathbf{m}]}. \quad (14)$$

The notation $\neg \mathbf{m}$ means the set of modes $m \notin \mathbf{m}$ (the complement of $\mathbf{m}$). The kernel matrix over the incomplete grid, $\hat{\mathbf{K}} \in \mathbb{R}^{\hat{N} \times \hat{N}}$, can be formally obtained from the complete-grid matrix, $\mathbf{K} \in \mathbb{R}^{N \times N}$, by appropriately deleting rows and columns. This can be written as

$$\hat{\mathbf{K}} = \mathbf{\Gamma}^T \mathbf{K} \mathbf{\Gamma} = \sum_{\mathbf{m} \in \mathcal{M}_{\text{kernel}}} \sigma_{|\mathbf{m}|}^2 \mathbf{\Gamma}^T \mathbf{K}^{[\mathbf{m}]} \mathbf{J}^{[\neg \mathbf{m}]} \mathbf{\Gamma}, \quad (15)$$

where $\mathbf{\Gamma} \in \mathbb{R}^{N \times \hat{N}}$ is a *chopping* matrix (the columns of $\mathbf{\Gamma}$ are the standard basis vectors $\mathbf{e}_{\mathbf{i}} \in \mathbb{R}^N$ with $\mathbf{i} \in \hat{\mathcal{I}}$). Equation (15) is thus a sum of chopped Kronecker products. It would seem that chopping destroys the structure of the complete-grid kernel, but we will see that chopping can in fact be handled efficiently and very elegantly if $\hat{\mathcal{I}}$ is CUD. Chopped Kronecker products have been used extensively for solving the vibrational Schrödinger equation [19–21], and the mathematical aspects have been formalized by Holzmüller and Pflüger [18] to whom we refer for details and additional references. We will use the following theorem:

Theorem 4.2. Let the index set $\widehat{\mathcal{I}}$ be CUD and let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{N \times N}$. In addition, let $\mathbf{M}^{[m]} \in \mathbb{R}^{N \times N}$ be an arbitrary one-mode bracket matrix (see Definition 4.1). Using hats to denote chopping, e.g. $\widehat{\mathbf{A}} = \Gamma^\top \mathbf{A} \Gamma$, we then have

- (i) If $\mathbf{A}$ is lower triangular or $\mathbf{B}$ is upper triangular, then $\widehat{\mathbf{AB}} = \widehat{\mathbf{A}}\widehat{\mathbf{B}}$.
- (ii) If $\mathbf{A}$ is lower (upper) triangular and invertible, then $\mathbf{A}^{-1}$ is lower (upper) triangular and $\widehat{\mathbf{A}}$ is invertible with $\widehat{\mathbf{A}}^{-1} = \widehat{\mathbf{A}^{-1}}$.
- (iii) The matrix-vector product $\widehat{\mathbf{M}^{[m]}}\widehat{\mathbf{v}} = \widehat{\mathbf{w}}$ can be computed with a complexity of at most $\mathcal{O}(n_m \widehat{N})$ without referencing the multi-indices $\mathbf{i} \notin \widehat{\mathcal{I}}$.

Proof. See Holzmüller and Pflüger [18, Theorems 1 and 3]. □

Theorem 4.2(ii) actually allows a separable kernel to be inverted directly over an incomplete grid, but only in the noiseless case (see Appendix E). Our initial investigations in this direction were not promising: (i) The complete lack of noise leads to very ill-conditioned numerics; (ii) the D -complexity is higher than CUTS-GPR; and (iii) a separable kernel is not attractive for the applications we have in mind.

The details of how to compute the MVP in Theorem 4.2(iii) are covered by Theorem E2 and Appendix F. The crucial aspect of Theorem 4.2(iii) is that it only ever references the multi-indices of the incomplete grid. The remaining multi-indices of the underlying complete grid, $\mathbf{i} \notin \widehat{\mathcal{I}}$, are never referenced, not even in intermediate steps. Staying on the incomplete grid at all times is a *necessary* condition for breaking the curse of dimensionality, but the complexity offered by Theorem 4.2(iii) is not *sufficient* in itself due the large number of terms in the additive kernel. This problem can, however, be resolved by careful use of structure and sparsity as described in Section 5.1.

5 Low-scaling implementation

5.1 Fast matrix-vector products

Our task is now to apply the chopping framework to the kernel matrix in (14). Before proceeding, we observe that the all-ones matrix can be factorized like

$$\mathbf{J}^{(m)} = \mathbf{L}^{(m)} \mathbf{R}^{(m)} \mathbf{U}^{(m)} \quad (16)$$

with

$$\mathbf{L}^{(m)} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 1 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 0 & \cdots & 1 \end{bmatrix}, \quad \mathbf{R}^{(m)} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}, \quad \mathbf{U}^{(m)} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}. \quad (17)$$

The matrix $\mathbf{R}^{(m)}$ is symmetric and idempotent and hence a projection. It is, more specifically, the matrix that projects onto the reference point for mode m . Using (16), a short derivation (see Appendix G.1) shows that (14) can be rewritten as

$$\mathbf{K} = \mathbf{L} \left[\sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \mathbf{M}^{[\mathbf{m}]} \mathbf{R}^{[\neg \mathbf{m}]} \right] \mathbf{U} \equiv \mathbf{L} \mathbf{M} \mathbf{U}, \quad \mathbf{M}^{(m)} = (\mathbf{L}^{(m)})^{-1} \mathbf{K}^{(m)} (\mathbf{U}^{(m)})^{-1}, \quad (18)$$

$$\mathbf{L} = \mathbf{L}^{(1)} \otimes \cdots \otimes \mathbf{L}^{(D)}, \quad \mathbf{U} = \mathbf{U}^{(1)} \otimes \cdots \otimes \mathbf{U}^{(D)}. \quad (19)$$

Each term in (18) has a lower triangular factor to the left ($\mathbf{L}$), a non-triangular factor in the middle ($\mathbf{M}^{[\mathbf{m}]}$) and two upper-triangular factors to the right ($\mathbf{R}^{[\neg \mathbf{m}]}$ and $\mathbf{U}$). We can therefore apply Theorem 4.2(i) thrice to get

$$\widehat{\mathbf{K}} = \widehat{\mathbf{L}} \left[\sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \widehat{\mathbf{M}^{[\mathbf{m}]}} \widehat{\mathbf{R}^{[\neg \mathbf{m}]}} \right] \widehat{\mathbf{U}} = \widehat{\mathbf{L}} \widehat{\mathbf{M}} \widehat{\mathbf{U}}. \quad (20)$$

Using the factorized form above, we can rephrase the MVP with $\widehat{\mathbf{K}}$ as a sequence of three simpler MVPs. The matrices $\widehat{\mathbf{L}} = \widehat{\mathbf{L}}^{[1]} \widehat{\mathbf{L}}^{[2]} \cdots \widehat{\mathbf{L}}^{[D]}$ and $\widehat{\mathbf{U}} = \widehat{\mathbf{U}}^{[1]} \widehat{\mathbf{U}}^{[2]} \cdots \widehat{\mathbf{U}}^{[D]}$ are evidently very sparse in addition to being highly structured. One can show that each factor touches only a small part of the vector, with the result that MVPs by $\widehat{\mathbf{L}}$ and $\widehat{\mathbf{U}}$ have a complexity of only $\mathcal{O}(\alpha \widehat{N})$ (see Appendix G.2). This should be compared to the naive complexity of $\mathcal{O}(nD\widehat{N})$ obtained by repeatedly applying Theorem 4.2(iii) without utilizing the special structure of the 1D matrices $\mathbf{L}^{(m)}$ and $\mathbf{U}^{(m)}$.

The MVPs with $\widehat{\mathbf{M}}$ is much more challenging, simply because of the large number of terms. However, we demonstrate in Appendix G.3 that each term is extremely sparse due to the presence of the projector $\widehat{\mathbf{R}}^{[-m]}$. This leads to a complexity of only $\mathcal{O}(n\alpha\widehat{N})$, which is also the complexity of the full MVP by $\widehat{\mathbf{K}}$. What we have obtained is a factorization of $\widehat{\mathbf{K}}$ into three factors that exhibit a highly structured kind of sparsity that allows an extremely fast MVP.

5.2 The quadratic term and overall cost

Computing the quadratic term of (4), $\boldsymbol{\alpha}^\top (\partial \widehat{\mathbf{C}} / \partial \theta) \boldsymbol{\alpha}$, is not a difficult task on its own, and the cost for each individual hyperparameter is usually not problematic. Consider, however, the task of computing *all* length scale quadratic terms. A typical implementation consists of a derivative MVP, $(\partial \widehat{\mathbf{C}} / \partial \ell^{(m)}) \boldsymbol{\alpha}$, followed by a dot product with $\boldsymbol{\alpha}$. Even neglecting the cost of the MVP, this leads to a complexity of $\mathcal{O}(D\widehat{N})$ due to the dot products alone. If the grid MCR is simple with cut level α , this implies a D -complexity of $\mathcal{O}(D^{\alpha+1})$ compared to $\mathcal{O}(D^\alpha)$ in the rest of the code. Appendix G.5 shows how to maintain a D -complexity of $\mathcal{O}(D^\alpha)$ by leveraging the factorized form of $\widehat{\mathbf{K}}$ in (20).

Appendix K gives an overview of the code and the cost of the various steps. The most expensive parts are the hyperparameter optimization and predictive variances with complexities $\mathcal{O}(N_{\text{opt}} N_{\text{probe}} N_{\text{CG}} (n\alpha\widehat{N} + k\widehat{N}))$ and $\mathcal{O}(\widehat{N}_* N_{\text{CG}} (n\alpha\widehat{N} + k\widehat{N}))$, respectively. The symbols N_{opt} , N_{probe} , N_{CG} and $\widehat{N}_*$ denote the number of optimization cycles, probe vectors (for trace estimation), conjugate gradient (CG) iterations and test points, respectively, while k is the preconditioner rank.

6 Numerical results

6.1 Computational complexity

The kernel MVP has a theoretical complexity of only $\mathcal{O}(n\alpha\widehat{N})$. If n and α are kept constant this translates to $\mathcal{O}(\widehat{N})$ or, equivalently, $\mathcal{O}(D^\alpha)$. It is important to verify this complexity and to assess the performance of the MVP in absolute terms. The D -complexity is determined for $\alpha = 2, 3, 4$ and $n = 5, 10, 20$ by running a series of simple benchmarks on a single core (see Appendix L for details and raw timings). The number of dimensions, D , is varied in the range from 16 to at most 16384 (depending on α and n). For each combination of α and n , the largest benchmark has $\widehat{N}$ equal to a few billion and a CPU time of no more than three minutes. CPU times have been fitted to a power law, $\log(t_{\text{CPU}}/\text{s}) = a \log(D) + b$, to determine the empirical D -complexity, $\mathcal{O}(D^\alpha)$. The fits are plotted for $\alpha = 2, 4$ in Figure 2a (Figure L2 shows $\alpha = 3$). The empirical slopes [2.06–2.08 ($\alpha = 2$), 3.09–3.15 ($\alpha = 3$) and 4.21–4.29 ($\alpha = 4$)] are slightly larger than theoretical, asymptotic slopes of 2, 3 and 4. This is simply a sign that the asymptotic region has not been fully reached, especially for $\alpha = 4$ (see also Appendix L). Figure 2b shows the CPU time for $\alpha = 2$ and $n = 10$ as a function of $\widehat{N}$, and the fit clearly verifies the linear $\widehat{N}$ -scaling.

6.2 Application to PES data

6.2.1 Computational setup

As a first large-scale application of CUTS-GPR, we consider the 24-dimensional PESs of the ten organic molecules listed in Table M1. The PESs were sampled according to the 1D grids in Table M2 with a cut level of $\alpha = 3$. The coarse grid ($n = 7$) was used for training ($\widehat{N} = 447\,265$), while the test set consists of the fine grid ($n = 11$) with the training points removed ($\widehat{N}_* = 1\,604\,576$).¹ We

¹The datasets are available at https://anonymous.4open.science/r/24D_3M_PES_DATA-F32E.

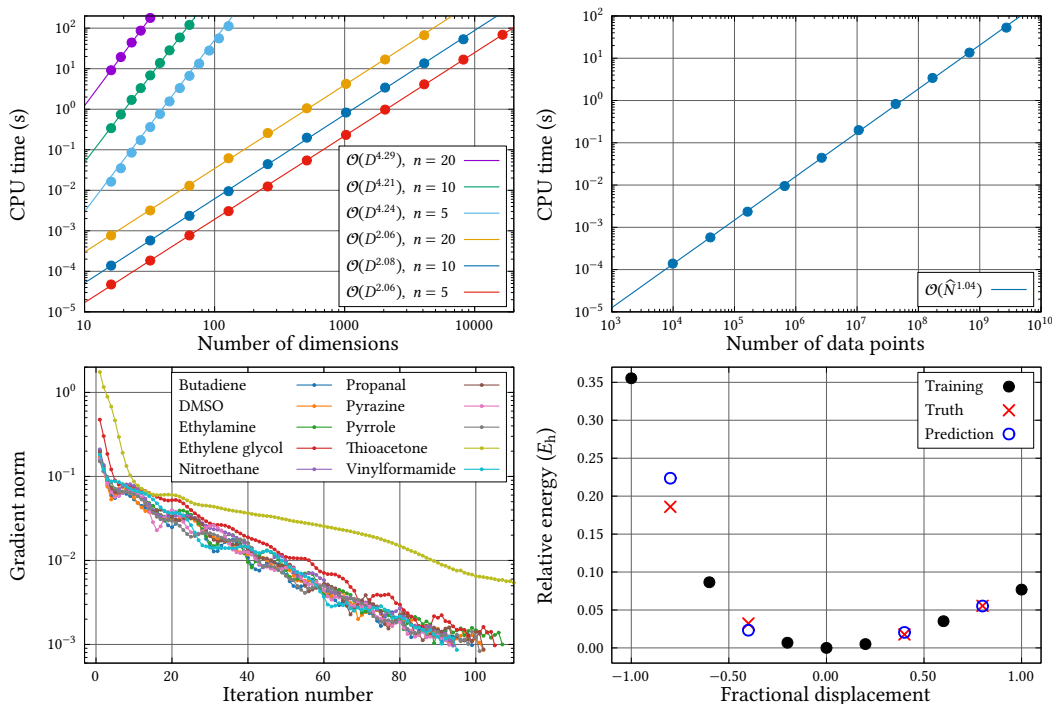


Figure 2: Top left (a): D -scaling of the kernel MVP for $\alpha = 2, 4$ and $n = 5, 10, 20$. Top right (b): $\tilde{N}$ -scaling of the kernel MVP for $\alpha = 2$ and $n = 10$. Bottom left (c): Learning curves for all ten molecules. Bottom right (d): Training data, predictions and reference data for the 24th 1D cut of the thioacetone PES.

used an OAK kernel with maximum interaction order $\omega = \alpha = 3$ (see Appendix M.2 for details on kernel centering). Before running the CUTS-GPR calculations, the training locations and training outputs were standardized to have zero mean and unit variance. All CUTS-GPR calculations were run with a fixed noise of $\sigma^2 = 10^{-3}$, while the remaining hyperparameters were optimized using the Adam algorithm [22] with learning rate 0.1 and a gradient norm threshold of $\varepsilon_{\text{opt}} = 10^{-3}$ (initial values are specified in Table M3). To regularize the hyperparameter optimization we place priors over σ_k^2 and $\ell^{(m)}$, similar to Lu et al. [15] and Hvarfner et al. [23] (see Appendix M.4 for details). The preconditioner rank was $k = 10$ and the CG residuals were converged to a relative norm of $\varepsilon_{\text{CG}} = 10^{-3}$. 35 reparameterized Rademacher probe vectors were used for stochastic trace estimation (see Appendices A.3 and A.4). All calculations were run on 36 cores of an AMD EPYC 9565 processor using standard double precision.

Few GPR methods (exact or approximate) can handle the combination of an additive kernel with large N and large D . We were therefore only able to compare against SVGP [8] as implemented in GPyTorch [11] with the OAK available in BoTorch [24]. The current BoTorch implementation only supports $\omega = 2$, so we used a locally modified version to also allow $\omega = 3$. Settings are described in Appendix M.5. The SVGP calculations were run on 36 cores of either an AMD EPYC 9565 CPU ($\omega = 3$) or a twin Intel Xeon Gold 6140 CPU ($\omega = 2$).

6.2.2 Results

First, we study the convergence of the hyperparameter optimization in CUTS-GPR. Figure 2c shows the learning curves for all ten molecules. The norm declines rather quickly during the first 5–10 iterations, after which the improvement slows down. All molecules converge after around 100 iterations, except for thioacetone, which requires 176 steps to reach the threshold. Although the gradient used for optimization is a stochastic estimate, convergence is steady and systematic. We attribute this to the fact that almost all trace estimates entering the gradient are remarkably well-determined with a standard error of the mean (SEM) less than 1% (see Table M4 for an example), even though the number of probe vectors (35) and the preconditioner rank (10) are not very

large. The exceptions are σ_0^2 and σ_1^2 , whose SEM are rather large (518 % and 37.6 %, respectively) (see Table M4). The large uncertainty on the σ_0^2 trace term does not appear to impair the overall convergence, something that we ascribe to the robustness of the Adam optimizer and to the fact that σ_0^2 is anyway very small.

Table M5 shows the CPU and wall times for hyperparameter optimization and predictions in CUTS-GPR (only the predictive mean was computed). The wall time spent on hyperparameter optimization ranges from 1.62 h to 3.58 h (mean 2.16 h), while predictions take less than two minutes for all 1 604 576 test points. This difference simply reflects the fact that the CG solver is run *many* times during optimization, while the predictive mean requires only a single run. Appendix M.8 contains the SVGP wall times. Even the fastest SVGP calculations (with $\omega = 2$) take around 40 min to complete. For $\omega = 3$, SVGP is always slower than CUTS-GPR. Many factors impact absolute timings, but we think it is fair to say that the cost of CUTS-GPR is comparable to or smaller than SVGP for the cases in question. This is remarkable, considering the fact that CUTS-GPR is numerically exact.

Figure 3 compares the maximum absolute error (MAX) and root mean square error (RMSE) for CUTS-GPR and SVGP (both error measures are range normalized and averaged over the ten molecules). The numerical values can be found in Appendix M.9, which also considers the mean absolute error (MAE). We find that CUTS-GPR outperforms SVGP for all three error measures (MAX, RMSE and MAE), which highlights the advantage of treating the kernel exactly. This is in fact true for each individual test case (see Appendix M.10). The difference between CUTS-GPR and SVGP is particularly large for the maximum error, which indicates that SVGP is not able to describe the most difficult points in spite of the fact that the target function is quite smooth. As seen in Figure 3, the SVGP errors exhibit diminishing returns with additional inducing points. Thus, increasing their number further is both impractical and unlikely to significantly improve accuracy. Even for CUTS-GPR, the maximum error is significantly larger than the MAE and RMSE. Large errors chiefly occur at points where the target function is very steep (see Figure 2d for an example), which is to be expected considering the relative coarseness of the training grid.

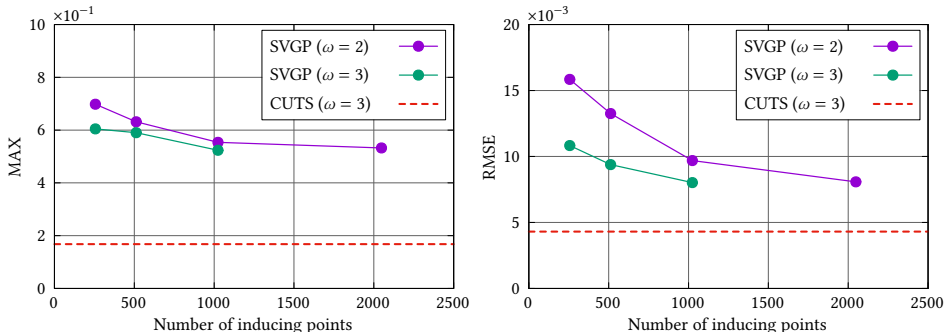


Figure 3: Comparison of range normalized MAX and RMSE. The errors are averaged over the ten molecules in Table M1.

7 Conclusions, limitations and extensions

We have introduced CUTS-GPR, a new approach for doing numerically exact GPR with large datasets in high-dimensional settings. CUTS-GPR is based on the combination of two attractive components: (i) Additive kernels and (ii) structured, incomplete grids. Careful analysis shows the surprising fact that this combination implies a highly structured kernel, which in turn allows a scalable kernel MVP. The MVP offers low-order polynomial scaling with D and near-linear or even linear scaling with N , something that we demonstrate theoretically and empirically. Without approximating the kernel matrix, CUTS-GPR enables GPR in previously inaccessible settings, potentially with billions of data points and thousands of dimensions – well beyond what is feasible with current methods.

In its current form, CUTS-GPR assumes a particular data format that may be too restrictive for some applications. Some restrictions can, however, be lifted with relative ease. Accidental missing data points can, for example, be handled by introducing an extra layer of chopping. Writing $\tilde{N} < \hat{N}$ for the actual number of data points, the kernel MVP then reads as $\tilde{\mathbf{C}}\tilde{\mathbf{v}} = \tilde{\mathbf{\Gamma}}^\top \hat{\mathbf{C}}\tilde{\mathbf{T}}\tilde{\mathbf{v}}$ with $\tilde{\mathbf{C}} \in \mathbb{R}^{\tilde{N} \times \tilde{N}}$.

In contrast to the chopping inside $\hat{\mathbf{C}}$, which is handled *implicitly*, the vector $\tilde{\mathbf{I}}\tilde{\mathbf{v}} \in \mathbb{R}^{\hat{N}}$ should be constructed *explicitly* followed by fast multiplication with $\hat{\mathbf{C}}$. We have also mentioned that the notion of an incomplete grid can be generalized by using the flexibility of the CUD property (Definition 3.3). This potentially allows a wider range of grid-type data sets to be addressed. One could also interpolate non-grid data onto an incomplete grid, similar to the idea of KISS-GP [9] whose applicability in high dimension is limited by the use of a complete grid.

Although the reliance on a particular data structure may be viewed as a limitation it should be remembered that, very often, the user controls the sampling of data and hence the data structure. In applications such as PES fitting, which we have considered in this paper, an incomplete grid structure is a very natural choice.

Computing predictive variances for very large test sets is a bottleneck in our current implementation. In the future, we plan on combining CUTS-GPR with Lanczos variance estimates (LOVE) [25] to enable much faster computation of variances.

Acknowledgments and Disclosure of Funding

This work was supported by the Danish National Research Foundation through the Center of Excellence for Chemistry of Clouds (Grant Agreement No. DNRF172) and by the Independent Research Fund Denmark through Grant No. 1026-00122B. The authors declare no competing interests.

References

- ¹Y. Saatchi, Scalable Inference for Structured Gaussian Process Models, PhD thesis (University of Cambridge, 2011).
- ²A. G. Wilson, A. Nehorai, E. Gilboa, and J. P. Cunningham, Fast Kernel Learning for Multidimensional Pattern Extrapolation, *Advances in Neural Information Processing Systems*, Vol. 27 (2014).
- ³E. Gilboa, Y. Saatchi, and J. P. Cunningham, Scaling Multidimensional Inference for Structured Gaussian Processes, *IEEE Trans. Pattern Anal. Mach. Intell.* **37**, 424–436 (2015) 10.1109/TPAMI.2013.192.
- ⁴S. Flaxman, A. G. Wilson, D. B. Neill, H. Nickisch, and A. J. Smola, Fast Kronecker Inference in Gaussian Processes with non-Gaussian Likelihoods, *Proceedings of the 32nd International Conference on Machine Learning* (2015).
- ⁵S. Ishida, F. Panero, and W. Bergsma, *Hierarchical additive interaction modelling with Gaussian process prior and its efficient implementation for multidimensional grid data*, 2025, 10.48550/arXiv.2305.07073.
- ⁶E. Snelson and Z. Ghahramani, Sparse Gaussian Processes using Pseudo-inputs, *Advances in Neural Information Processing Systems*, Vol. 18 (2005).
- ⁷M. Titsias, Variational learning of inducing variables in sparse gaussian processes, *Proceedings of the twelfth international conference on artificial intelligence and statistics*, Vol. 5, *Proceedings of Machine Learning Research* (2009).
- ⁸J. Hensman, N. Fusi, and N. D. Lawrence, Gaussian Processes for Big Data, *Proceedings of the Twenty-Ninth Conference on Uncertainty in Artificial Intelligence* (2013), pp. 282–290.
- ⁹A. G. Wilson and H. Nickisch, Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP), *Proceedings of the 32nd International Conference on Machine Learning*, Vol. 37 (2015).
- ¹⁰J. Gardner, G. Pleiss, R. Wu, K. Weinberger, and A. Wilson, Product Kernel Interpolation for Scalable Gaussian Processes, *Proceedings of the 21st International Conference on Artificial Intelligence and Statistics* (2018), pp. 1407–1416.
- ¹¹J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger, and A. G. Wilson, GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration, *Advances in Neural Information Processing Systems* 31 (2018).
- ¹²J. Wenger, G. Pleiss, P. Hennig, J. P. Cunningham, and J. R. Gardner, Preconditioning for Scalable Gaussian Process Hyperparameter Optimization, *Proceedings of the 39th International Conference on Machine Learning* (2022).
- ¹³D. K. Duvenaud, H. Nickisch, and C. E. Rasmussen, Additive Gaussian Processes, *Advances in Neural Information Processing Systems*, Vol. 24 (2011).
- ¹⁴N. Durrande, D. Ginsbourger, and L. Carraro, Additive covariance kernels for high-dimensional gaussian process modeling, *Annales de la faculté des sciences de Toulouse Mathématiques* **21**, 481–499 (2012) 10.48550/arXiv.1111.6233.
- ¹⁵X. Lu, A. Boukouvalas, and J. Hensman, Additive Gaussian Processes Revisited, *Proceedings of the 39th International Conference on Machine Learning* (2022).
- ¹⁶T. G. Kolda and B. W. Bader, Tensor Decompositions and Applications, *SIAM Rev.* **51**, 455–500 (2009) 10.1137/07070111X.
- ¹⁷O. Christiansen, A second quantization formulation of multimode dynamics, *J. Chem. Phys.* **120**, 2140–2148 (2004) 10.1063/1.1637578.
- ¹⁸D. Holzmüller and D. Pflüger, Fast Sparse Grid Operations Using the Unidirectional Principle: A Generalized and Unified Framework, *Sparse Grids and Applications – Munich 2018*, edited by H.-J. Bungartz, J. Garcke, and D. Pflüger (2021), pp. 69–100, 10.1007/978-3-030-81362-8_4.
- ¹⁹R. Wodraszka and T. Carrington, A pruned collocation-based multiconfiguration time-dependent Hartree approach using a Smolyak grid for solving the Schrödinger equation with a general potential energy surface, *J. Chem. Phys.* **150**, 154108 (2019) 10.1063/1.5093317.
- ²⁰E. J. Zak and T. Carrington, Using collocation and a hierarchical basis to solve the vibrational Schrödinger equation, *The Journal of Chemical Physics* **150**, 204108 (2019) 10.1063/1.5096169.

- ²¹J. Simmons and T. Carrington, Computing vibrational spectra using a new collocation method with a pruned basis and more points than basis functions: Avoiding quadrature, *J. Chem. Phys.* **158**, 144115 (2023) 10.1063/5.0146703.
- ²²D. P. Kingma and J. Ba, Adam: A Method for Stochastic Optimization, Proceedings of the 3rd International Conference on Learning Representations (2015), 10.48550/arXiv.1412.6980.
- ²³C. Hvarfner, E. O. Hellsten, and L. Nardi, Vanilla Bayesian Optimization Performs Great in High Dimensions, Proceedings of the 41st International Conference on Machine Learning (2024), 10.48550/arXiv.2402.02229.
- ²⁴M. Balandat et al., BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization, *Advances in Neural Information Processing Systems*, Vol. 33 (2020).
- ²⁵G. Pleiss, J. R. Gardner, K. Q. Weinberger, and A. G. Wilson, Constant-Time Predictive Distributions for Gaussian Processes, Proceedings of the 35th International Conference on Machine Learning (2018).
- ²⁶M. Hestenes and E. Stiefel, Methods of conjugate gradients for solving linear systems, *J. Res. Natl. Bur. Stan.* **49**, 409 (1952) 10.6028/jres.049.044.
- ²⁷G. Meurant and Z. Strakoš, The Lanczos and conjugate gradient algorithms in finite precision arithmetic, *Acta Numerica* **15**, 471–542 (2006) 10.1017/S096249290626001X.
- ²⁸Y. Saad, *Iterative methods for sparse linear systems*, 2. ed. (SIAM, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2007).
- ²⁹N. H. F. Beebe and J. Linderberg, Simplifications in the generation and transformation of two-electron integrals in molecular calculations, *Int. J. Quantum Chem.* **12**, 683–705 (1977) 10.1002/qua.560120408.
- ³⁰H. Harbrecht, M. Peters, and R. Schneider, On the low-rank approximation by the pivoted Cholesky decomposition, *Appl. Numer. Math.* **62**, 428–440 (2012) 10.1016/j.apnum.2011.10.001.
- ³¹Y. Chen, E. N. Epperly, J. A. Tropp, and R. J. Webber, Randomly pivoted Cholesky: Practical approximation of a kernel matrix with few entry evaluations, *Communications on Pure and Applied Mathematics* **78**, 995–1041 (2025) 10.1002/cpa.22234.
- ³²C. Lanczos, An iteration method for the solution of the eigenvalue problem of linear differential and integral operators, *J. Res. Natl. Bur. Stan.* **45**, 255 (1950) 10.6028/jres.045.026.
- ³³T. Chen, *The Lanczos algorithm for matrix functions: a handbook for scientists* (2024), 10.48550/arXiv.2410.11090.
- ³⁴C. Musco, C. Musco, and A. Sidford, *Stability of the Lanczos Method for Matrix Function Approximation*, 2024, 10.48550/arXiv.1708.07788.
- ³⁵A. Girard, A fast ‘Monte-Carlo cross-validation’ procedure for large least squares problems with noisy data, *Numer. Math.* **56**, 1–23 (1989) 10.1007/BF01395775.
- ³⁶M. Hutchinson, A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines, *Communications in Statistics - Simulation and Computation* **18**, 1059–1076 (1989) 10.1080/03610918908812806.
- ³⁷H.-J. Bungartz and M. Griebel, Sparse grids, *Acta Numer.* **13**, 147–269 (2004) 10.1017/S0962492904000182.
- ³⁸P. Seidler, M. B. Hansen, and O. Christiansen, Towards fast computations of correlated vibrational wave functions: Vibrational coupled cluster response excitation energies at the two-mode coupling level, *J. Chem. Phys.* **128**, 154113 (2008) 10.1063/1.2907860.
- ³⁹P. Seidler and O. Christiansen, Automatic derivation and evaluation of vibrational coupled cluster theory equations, *J. Chem. Phys.* **131**, 234109 (2009) 10.1063/1.3272796.
- ⁴⁰F. Aquilante, R. Lindh, and T. B. Pedersen, Analytic derivatives for the Cholesky representation of the two-electron integrals, *The Journal of Chemical Physics* **129**, 034106 (2008) 10.1063/1.2955755.
- ⁴¹X. Feng, E. Epifanovsky, J. Gauss, and A. I. Krylov, Implementation of analytic gradients for CCSD and EOM-CCSD using Cholesky decomposition of the electron-repulsion integrals and their derivatives: Theory and benchmarks, *The Journal of Chemical Physics* **151**, 014110 (2019) 10.1063/1.5100022.

- ⁴²S. P. Smith, Differentiation of the Cholesky Algorithm, *Journal of Computational and Graphical Statistics* **4**, 134–147 (1995) 10.1080/10618600.1995.10474671.
- ⁴³C. Bannwarth, S. Ehlert, and S. Grimme, GFN2-xTB—An Accurate and Broadly Parametrized Self-Consistent Tight-Binding Quantum Chemical Method with Multipole Electrostatics and Density-Dependent Dispersion Contributions, *J. Chem. Theory Comput.* **15**, 1652–1671 (2019) 10.1021/acs.jctc.8b01176.
- ⁴⁴C. Bannwarth et al., Extended tight-binding quantum chemistry methods, *WIREs Comput. Mol. Sci.* **11**, e1493 (2021) 10.1002/wcms.1493.
- ⁴⁵O. Christiansen et al., *MidasCpp (Molecular Interactions, Dynamics and Simulation Chemistry Program Package)*, Aarhus University, 2025.

A Gaussian process regression using matrix-vector products

A.1 Conjugate gradient

Linear systems like $\mathbf{C}\mathbf{x} = \mathbf{b}$ can be solved iteratively using the well-known CG algorithm [26–28]. Since the overall computational effort scales linearly with the number of CG iterations, it is important to accelerate convergence as much as possible. This can be achieved by using a preconditioner, i.e. a symmetric positive definite (SPD) matrix $\mathbf{P} \approx \mathbf{C}$ that allows fast solves, $\mathbf{P}^{-1}\mathbf{v}$. The preconditioner is formally introduced into the CG algorithm by using its Cholesky decomposition $\mathbf{P} = \mathbf{E}\mathbf{E}^\top$ to write an equivalent linear system,

$$(\mathbf{E}^{-1}\mathbf{C}\mathbf{E}^{-\top})(\mathbf{E}^\top\mathbf{x}) = (\mathbf{E}^{-1}\mathbf{b}) \quad (\text{A1})$$

This system has the same solution as the original system, but convergence depends on the conditioning of the matrix $\tilde{\mathbf{C}} = \mathbf{E}^{-1}\mathbf{C}\mathbf{E}^{-\top}$ rather than $\mathbf{C}$ itself. If $\mathbf{P}$ approximates $\mathbf{C}$ well then $\tilde{\mathbf{C}} \approx \mathbf{I}$ and convergence can be much improved. This is often summarized in terms of condition numbers:

$$\kappa_{\text{precond}} = \frac{\lambda_{\max}(\tilde{\mathbf{C}})}{\lambda_{\min}(\tilde{\mathbf{C}})} \ll \frac{\lambda_{\max}(\mathbf{C})}{\lambda_{\min}(\mathbf{C})} = \kappa_{\text{orig}}. \quad (\text{A2})$$

The Cholesky factor $\mathbf{E}$ is not needed in the actual implementation of the algorithm and serves only as a formal device. All that is needed is MVPs with the inverse preconditioner, $\mathbf{P}^{-1}\mathbf{v}$.

Any convenient preconditioner can be used, but we will rely on a rank- k pivoted Cholesky decomposition [29–31] such that

$$\mathbf{Z}\mathbf{Z}^\top \approx \mathbf{K}, \quad \mathbf{Z} \in \mathbb{R}^{N \times k} \quad (\text{A3})$$

$$\mathbf{P} = \mathbf{Z}\mathbf{Z}^\top + \sigma^2\mathbf{I} \approx \mathbf{C}. \quad (\text{A4})$$

From the matrix inverse and matrix determinant lemmas we get the following simple expressions:

$$\mathbf{P}^{-1} = \frac{1}{\sigma^2} (\mathbf{I}_N - \mathbf{Z}\mathbf{Q}^{-1}\mathbf{Z}^\top), \quad (\text{A5})$$

$$\log |\mathbf{P}| = \log |\mathbf{Q}| + (N - k) \log(\sigma^2), \quad (\text{A6})$$

$$\mathbf{Q} = \sigma^2\mathbf{I}_k + \mathbf{Z}^\top\mathbf{Z} \in \mathbb{R}^{k \times k}. \quad (\text{A7})$$

The matrix $\mathbf{Q}$ and its Cholesky factorization can be computed in $\mathcal{O}(k^2N)$ and $\mathcal{O}(k^3)$ time, respectively. Once this is done, the cost of $\log |\mathbf{P}|$ is only $\mathcal{O}(k)$ while $\mathbf{P}^{-1}\mathbf{v}$ has a complexity of $\mathcal{O}(kN)$.

The pivoted Cholesky algorithm evaluates the main diagonal and k columns of $\mathbf{K}$ at a cost of $\mathcal{O}(\rho_{\text{diag}})$ and $\mathcal{O}(k\rho_{\text{col}})$, respectively (ρ_{diag} and ρ_{col} depend on the structure of the kernel; see Appendices H and I). In addition, the algorithm expends $\mathcal{O}(k^2N)$ arithmetic operations and requires $\mathcal{O}(kN)$ storage [31].

A.2 Lanczos

The (non-preconditioned) Lanczos algorithm [32, 33] can be described very informally as follows: Running ℓ iterations of the Lanczos algorithm with a symmetric matrix $\mathbf{C}$ and starting vector $\mathbf{b}$ builds an approximation

$$\mathbf{C} \approx \mathbf{Q}_\ell \mathbf{T}_\ell \mathbf{Q}_\ell^\top, \quad \mathbf{T}_\ell \in \mathbb{R}^{\ell \times \ell}, \quad \mathbf{Q}_\ell \in \mathbb{R}^{N \times \ell}. \quad (\text{A8})$$

The matrix $\mathbf{T}_\ell$ is symmetric tridiagonal and the columns of $\mathbf{Q}_\ell$ (the Lanczos vectors) are orthonormal (in exact arithmetic) with the first column being $\mathbf{q}_0 = \mathbf{b}/\|\mathbf{b}\|$. It follows that

$$\mathbf{Q}_\ell^\top \mathbf{b} = \mathbf{e}_0 \|\mathbf{b}\|. \quad (\text{A9})$$

Lanczos can be used to approximate quadratic forms involving matrix functions, a technique known as Lanczos quadrature [33]:

$$\begin{aligned} \mathbf{b}^\top f(\mathbf{C}) \mathbf{b} &\approx \mathbf{b}^\top \mathbf{Q}_\ell f(\mathbf{T}_\ell) \mathbf{Q}_\ell^\top \mathbf{b} \\ &= \|\mathbf{b}\|^2 \mathbf{e}_0^\top f(\mathbf{T}_\ell) \mathbf{e}_0. \end{aligned} \quad (\text{A10})$$

Although the equality due to (A9) only holds in exact arithmetic, one should always use the latter expression [34]. This has the added benefit that the Lanczos vectors need not be stored.

The Lanczos algorithm (with starting vector $\mathbf{b}$) and the CG algorithm (with right-hand side $\mathbf{b}$ and zero starting guess) are intimately related, and the quantities of one can be obtained from the other [27, 28]. With a slight (and very inexpensive) modification of CG [11] we can thus obtain the Lanczos tridiagonal matrix $\mathbf{T}_\ell$ and compute

$$\mathbf{b}^\top \log(\mathbf{C})\mathbf{b} \approx \|\mathbf{b}\|^2 \mathbf{e}_0^\top \log(\mathbf{T}_\ell) \mathbf{e}_0. \quad (\text{A11})$$

The matrix logarithm of $\mathbf{T}_\ell$ can be computed by diagonalization, which has a cost of $\mathcal{O}(\ell^2)$ since $\mathbf{T}_\ell$ is symmetric tridiagonal.

If a preconditioner is used, the equations above must be modified slightly. In this case the Lanczos algorithm (and the modified CG algorithm) build an approximation of the preconditioned matrix $\tilde{\mathbf{C}} = \mathbf{E}^{-1}\mathbf{C}\mathbf{E}^{-\top}$ and the formal starting vector is the preconditioned vector $\tilde{\mathbf{b}} = \mathbf{E}^{-1}\mathbf{b}$ (rather the user-supplied vector $\mathbf{b}$). Comparing with (A11) we thus find that

$$\tilde{\mathbf{b}}^\top \log(\tilde{\mathbf{C}})\tilde{\mathbf{b}} \approx \|\tilde{\mathbf{b}}\|^2 \mathbf{e}_0^\top \log(\tilde{\mathbf{T}}_\ell) \mathbf{e}_0 \quad (\text{A12})$$

or, equivalently,

$$\mathbf{b}^\top \mathbf{E}^{-\top} \log(\tilde{\mathbf{C}}) \mathbf{E}^{-1} \mathbf{b} \approx (\mathbf{b}^\top \mathbf{P}^{-1} \mathbf{b}) \mathbf{e}_0^\top \log(\tilde{\mathbf{T}}_\ell) \mathbf{e}_0. \quad (\text{A13})$$

Importantly, the right-hand side of (A13) does not refer to $\mathbf{E}$, which can once again be considered as a formal device.

A.3 Stochastic trace estimation

The trace of a matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ can be computed as the expectation value of a quadratic form. To see this, we consider a random vector $\mathbf{b} \in \mathbb{R}^N$ with $\mathbb{E}[\mathbf{b}\mathbf{b}^\top] = \mathbf{I}$, e.g. a Rademacher random vector. Using the cyclic property of the trace and the linearity of the expectation value, we find that

$$\begin{aligned} \mathbb{E}[\mathbf{b}^\top \mathbf{A} \mathbf{b}] &= \mathbb{E}[\text{tr}(\mathbf{b}^\top \mathbf{A} \mathbf{b})] \\ &= \mathbb{E}[\text{tr}(\mathbf{A} \mathbf{b} \mathbf{b}^\top)] \\ &= \text{tr}(\mathbf{A} \mathbb{E}[\mathbf{b} \mathbf{b}^\top]) \\ &= \text{tr}(\mathbf{A}). \end{aligned} \quad (\text{A14})$$

This observation leads to the Girard–Hutchinson estimator [35, 36] with m probe vectors:

$$\text{tr}(\mathbf{A}) \approx \frac{1}{m} \sum_{i=1}^m \mathbf{b}_i^\top \mathbf{A} \mathbf{b}_i. \quad (\text{A15})$$

Due to the use of a preconditioner, we must modify the trace estimation slightly and use a random vector with $\mathbb{E}[\mathbf{b}\mathbf{b}^\top] = \mathbf{P}$ (Section A.4 shows how to sample such a vector). Using the same arguments that lead to (A14) we see that

$$\text{tr}(\mathbf{A}) = \mathbb{E}[\mathbf{b}^\top \mathbf{A} \mathbf{P}^{-1} \mathbf{b}] \quad (\text{A16})$$

$$= \mathbb{E}[\mathbf{b}^\top \mathbf{E}^{-\top} \mathbf{A} \mathbf{E}^{-1} \mathbf{b}]. \quad (\text{A17})$$

Equation (A16) is used to estimate the trace term,

$$\begin{aligned} \text{tr} \left(\mathbf{C}^{-1} \frac{\partial \mathbf{C}}{\partial \theta} \right) &= \mathbb{E}[\mathbf{b}^\top \mathbf{C}^{-1} \frac{\partial \mathbf{C}}{\partial \theta} \mathbf{P}^{-1} \mathbf{b}] \\ &\approx \frac{1}{m} \sum_{i=1}^m \mathbf{b}_i^\top \mathbf{C}^{-1} \frac{\partial \mathbf{C}}{\partial \theta} \mathbf{P}^{-1} \mathbf{b}_i, \end{aligned} \quad (\text{A18})$$

while (A17) and (A13) result in an estimate for the log-determinant of $\tilde{\mathbf{C}}$:

$$\begin{aligned} \log |\tilde{\mathbf{C}}| &= \text{tr} \log(\tilde{\mathbf{C}}) \\ &= \mathbb{E}[\mathbf{b}^\top \mathbf{E}^{-\top} \log(\tilde{\mathbf{C}}) \mathbf{E}^{-1} \mathbf{b}] \\ &\approx \frac{1}{m} \sum_{i=1}^m \mathbf{b}_i^\top \mathbf{E}^{-\top} \log(\tilde{\mathbf{C}}) \mathbf{E}^{-1} \mathbf{b}_i \\ &\approx \frac{1}{m} \sum_{i=1}^m (\mathbf{b}_i^\top \mathbf{P}^{-1} \mathbf{b}_i) \mathbf{e}_0^\top \log(\tilde{\mathbf{T}}_{\ell,i}) \mathbf{e}_0. \end{aligned} \quad (\text{A19})$$

It is important to note that (A18) uses stochastic trace estimation alone, while (A19) involves two separate approximations, namely stochastic trace estimation and Lanczos quadrature (this combination is called stochastic Lanczos quadrature).

For each probe vector $\mathbf{b}_i$ a single run of the modified, preconditioned CG algorithm produces the linear solve $\mathbf{C}^{-1}\mathbf{b}_i$ used in (A18) as well as the Lanczos tridiagonal matrix $\tilde{\mathbf{T}}_{\ell,i}$ used in (A19). The preconditioner solve $\mathbf{P}^{-1}\mathbf{b}_i$ is used in both estimates and is easily obtained from (A5) as already explained.

Having computed an estimate of $\log |\tilde{\mathbf{C}}|$, we need to recover an estimate of $\log |\mathbf{C}|$. Using the fact that $\mathbf{C}$ and $\mathbf{P}$ have positive determinants (since they are SPD) we get

$$\begin{aligned}\log |\tilde{\mathbf{C}}| &= \log |\mathbf{E}^{-1}\mathbf{C}\mathbf{E}^{-\top}| = \log |\mathbf{C}\mathbf{P}^{-1}| \\ &= \log |\mathbf{C}| - \log |\mathbf{P}|\end{aligned}\tag{A20}$$

or simply

$$\log |\mathbf{C}| = \log |\mathbf{P}| + \log |\tilde{\mathbf{C}}|.\tag{A21}$$

The estimated log-determinant of $\mathbf{C}$ can thus be expressed as the sum of two terms: A deterministic approximation based on the preconditioner and a stochastic estimate of the remainder. If the preconditioner is sufficiently good, the remainder will be small and the associated stochastic noise will also be small. This reduces the variance of the overall estimate of $\log |\mathbf{C}|$ and the MLL [12], which is very helpful when optimizing hyperparameters.

It is possible to achieve a similar variance reduction for the trace term by simply adding and subtracting a preconditioner term:

$$\text{tr} \left(\mathbf{C}^{-1} \frac{\partial \mathbf{C}}{\partial \theta} \right) = \text{tr} \left(\mathbf{P}^{-1} \frac{\partial \mathbf{P}}{\partial \theta} \right) + \left[\text{tr} \left(\mathbf{C}^{-1} \frac{\partial \mathbf{C}}{\partial \theta} \right) - \text{tr} \left(\mathbf{P}^{-1} \frac{\partial \mathbf{P}}{\partial \theta} \right) \right]\tag{A22}$$

The first term on the right-hand side is computed deterministically (via a closed-form expression), while the term in square brackets is estimated by using (A18).

A.4 Sampling the probe vectors

Consider two independent random vectors $\mathbf{v} \in \mathbb{R}^k$ and $\mathbf{w} \in \mathbb{R}^N$ such that

$$\mathbb{E}[\mathbf{v}\mathbf{v}^\top] = \mathbf{I}_k, \quad \boldsymbol{\mu} = \mathbb{E}[\mathbf{v}] = 0,\tag{A23a}$$

$$\mathbb{E}[\mathbf{w}\mathbf{w}^\top] = \mathbf{I}_N, \quad \boldsymbol{\nu} = \mathbb{E}[\mathbf{w}] = 0.\tag{A23b}$$

In this paper, $\mathbf{v}$ and $\mathbf{w}$ are Rademacher random vectors. Since the vectors are independent with mean zero we have

$$\mathbf{0} = \text{cov}(\mathbf{v}, \mathbf{w}) = \mathbb{E}[(\mathbf{v} - \boldsymbol{\mu})(\mathbf{w} - \boldsymbol{\nu})^\top] = \mathbb{E}[\mathbf{v}\mathbf{w}^\top].\tag{A24}$$

Now let

$$\mathbf{b} = \mathbf{Z}\mathbf{v} + \sigma\mathbf{w}, \quad \mathbf{Z} \in \mathbb{R}^{N \times k}.\tag{A25}$$

Expanding the terms and using (A23) and (A24) one easily finds that

$$\mathbb{E}[\mathbf{b}\mathbf{b}^\top] = \mathbf{Z}\mathbf{Z}^\top + \sigma^2\mathbf{I} = \mathbf{P}.\tag{A26}$$

B Properties of Kronecker products

The following theorem states a few important properties of the Kronecker product.

Theorem B.1. Let $\mathbf{A}^{(m)}, \mathbf{B}^{(m)} \in \mathbb{R}^{n_m \times n_m}$ for $m = 1, 2, \dots, D$.

(i) The Kronecker product is linear in each factor (multilinear):

$$\begin{aligned} \mathbf{A}^{(1)} \otimes \dots \otimes (\mathbf{A}^{(m)} + \mathbf{B}^{(m)}) \otimes \dots \otimes \mathbf{A}^{(D)} &= \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(m)} \otimes \dots \otimes \mathbf{A}^{(D)} \\ &+ \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{B}^{(m)} \otimes \dots \otimes \mathbf{A}^{(D)}. \end{aligned}$$

(ii) The Kronecker product is compatible with the ordinary matrix product (mixed-product property):

$$(\mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(D)}) (\mathbf{B}^{(1)} \otimes \dots \otimes \mathbf{B}^{(D)}) = (\mathbf{A}^{(1)} \mathbf{B}^{(1)}) \otimes \dots \otimes (\mathbf{A}^{(D)} \mathbf{B}^{(D)}).$$

(iii) A Kronecker product is invertible if each factor is invertible. In that case

$$(\mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(D)})^{-1} = (\mathbf{A}^{(1)})^{-1} \otimes \dots \otimes (\mathbf{A}^{(D)})^{-1}.$$

(iv) Matrix-vector products with $\mathbf{A} = \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(D)}$ can be written as a sequence of one-index contractions:

$$\begin{aligned} (\mathbf{A} \mathbf{v})_{\mathbf{i}} &= \sum_{\mathbf{j}} A_{\mathbf{i}\mathbf{j}} v_{\mathbf{j}} \\ &= \sum_{j_1} A_{i_1 j_1}^{(1)} \sum_{j_2} A_{i_2 j_2}^{(2)} \dots \sum_{j_D} A_{i_D j_D}^{(D)} v_{i_1 i_2 \dots i_D}. \end{aligned}$$

(v) Matrix-vector product with a one-mode bracket matrix can be written as a single one-index contraction:

$$\begin{aligned} (\mathbf{A}^{[m]} \mathbf{v})_{\mathbf{i}} &= \sum_{\mathbf{j}} A_{\mathbf{i}\mathbf{j}}^{[m]} v_{\mathbf{j}} \\ &= \sum_{j_m} A_{i_m j_m}^{(m)} v_{i_1 \dots i_{m-1} j_m i_{m+1} \dots i_D}. \end{aligned}$$

(vi) Assume that the matrices $\mathbf{A}^{(m)}$ are diagonalizable as $\mathbf{A}^{(m)} = \mathbf{Q}^{(m)} \mathbf{D}^{(m)} (\mathbf{Q}^{(m)})^{-1}$. Then the Kronecker product $\mathbf{A} = \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(D)}$ is also diagonalizable:

$$\begin{aligned} \mathbf{A} &= \mathbf{Q} \mathbf{D} \mathbf{Q}^{-1}, \\ \mathbf{Q} &= \mathbf{Q}^{(1)} \otimes \dots \otimes \mathbf{Q}^{(D)}, \\ \mathbf{D} &= \mathbf{D}^{(1)} \otimes \dots \otimes \mathbf{D}^{(D)}. \end{aligned}$$

(vii) The bracket matrices commute in the sense that

$$\begin{aligned} [\mathbf{A}^{[\mathbf{m}]}, \mathbf{A}^{[\mathbf{m}']}] &= 0 \quad \text{if } \mathbf{m} \neq \mathbf{m}', \\ [\mathbf{A}^{[\mathbf{m}]}, \mathbf{A}^{[\mathbf{m}']}] &= 0 \quad \text{if } \mathbf{m} \cap \mathbf{m}' = \emptyset. \end{aligned}$$

Proof. The statements (i)–(v) are easily verified using Definition 4.1(i). Statements (vi) and (vii) follow directly from (ii). $\square$

B.1 Implications for complete-grid GPR

Since a Kronecker product can be diagonalized in closed form via Theorem B.1(vi), this allows spherical noise to be easily added while maintaining structure:

$$\mathbf{C} = \mathbf{K} + \sigma^2 \mathbf{I} = \mathbf{Q} \mathbf{D} \mathbf{Q}^T + \sigma^2 \mathbf{I} = \mathbf{Q} (\mathbf{D} + \sigma^2 \mathbf{I}) \mathbf{Q}^T, \quad (\text{B1})$$

$$\mathbf{Q} = \mathbf{Q}^{(1)} \otimes \dots \otimes \mathbf{Q}^{(D)}, \quad \mathbf{D} = \mathbf{D}^{(1)} \otimes \dots \otimes \mathbf{D}^{(D)}, \quad (\mathbf{Q}^{(m)})^T \mathbf{K}^{(m)} \mathbf{Q}^{(m)} = \mathbf{D}^{(m)}. \quad (\text{B2})$$

Noting the Kronecker structure of the eigenvector matrix, Theorem B.1(iv) leads to inverse kernel MVPs (e.g. weights) with a complexity of only $\mathcal{O}((n_1 + \dots + n_D)N) = \mathcal{O}(nDN)$ (assuming 1D grids of equal size, n). Trace terms and log-determinants can be obtained with even less computational effort [see 1, Chapter 5 for details].

C Formal definition of subgrids and incomplete grids

Definition C.1. The subgrid $\mathcal{I}^{(\mathbf{m})}$ with displaced dimensions $\mathbf{m} = (m_1, m_2, \dots, m_k)$ and dimension $k = |\mathbf{m}|$ is the set of multi-indices of the form

$$\mathbf{i} = (i_1, i_2, \dots, i_D), \quad i_m = \begin{cases} 0 & \text{if } m \notin \mathbf{m}, \\ a_m & \text{if } m \in \mathbf{m}, \end{cases}$$

where $0 < a_m < n_m$. Recording the non-zero indices in a vector $\mathbf{a} = (a_{m_1}, a_{m_2}, \dots, a_{m_k})$ and making the association $\mathbf{i} \cong (\mathbf{m}, \mathbf{a})$, we may equivalently define $\mathcal{I}^{(\mathbf{m})}$ as the set of tuples

$$(\mathbf{m}, \mathbf{a}), \quad \mathbf{a} \in \mathcal{A}^{(\mathbf{m})} = \mathcal{A}^{(m_1)} \times \mathcal{A}^{(m_2)} \times \dots \times \mathcal{A}^{(m_k)},$$

where $\mathcal{A}^{(m)} = \mathcal{I}^{(m)} \setminus \{0\}$ is the set of non-zero indices for dimension m . Generally,

$$|\mathcal{I}^{(\mathbf{m})}| = \prod_{l=1}^k |\mathcal{A}^{(m_l)}| = \prod_{l=1}^k (|\mathcal{I}^{(m_l)}| - 1). \quad (\text{C1})$$

For later use, we note that $\mathcal{I}^{(\emptyset)}$ is the set containing the reference alone.

With this in place, the following definition follows naturally:

Definition C.2. A cut-based incomplete grid is a set of points $\hat{\mathcal{I}} \subseteq \mathcal{I}$ of the form

$$\hat{\mathcal{I}} = \bigcup_{\mathbf{m} \in \mathcal{M}_{\text{grid}}} \mathcal{I}^{(\mathbf{m})}. \quad (\text{C2})$$

The set $\mathcal{M}_{\text{grid}}$ specifies the cuts or subgrids that are included in $\hat{\mathcal{I}}$.

Definition C.2 suggests a certain data structure for vectors indexed by $\hat{\mathcal{I}}$ (e.g. the vector of observed data, $\hat{\mathbf{y}}$): The reference data point is a scalar, the 1D data is organized as a set of vectors, the 2D data as a set of matrices and so on. In short, a low-dimensional *subgrid*, $\mathcal{I}^{(\mathbf{m})}$, translates to a low-dimensional *subtensor*, $\mathbf{y}^{(\mathbf{m})}$ (cf. Example 3.1, Figure 1 and Figure C.1).

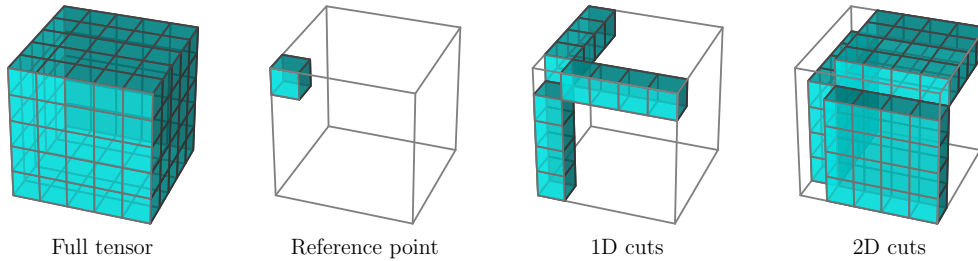


Figure C.1: Subtensors corresponding to the subgrids in Example 3.1.

D A generalized notion of CUD

It turns out that the version of the CUD property given in the main text (Definition 3.3) can be generalized considerably, as described by Holzmüller and Pflüger [18]. Here, we state a few selected definitions from their work that indicate how our GPR approach can be extended to other types of grids. The framework of Holzmüller and Pflüger [18] relies heavily on the notion of a preorder:

Definition D.1. A preorder is a binary relation $\leq$ on a set $\mathcal{S}$ that satisfies (i) and (ii) for all $i, j, k \in \mathcal{S}$:

- (i) Reflexivity: $i \leq i$ (every element is related to itself).
- (ii) Transitivity: $i \leq j$ and $j \leq k$ implies $i \leq k$.
- (iii) Antisymmetry: $i \leq j$ and $j \leq i$ implies $i = j$ (no two distinct elements precede each other).
- (iv) Strong connectedness: $i \leq j$ or $j \leq i$ (all pairs of elements are related).

A partial order satisfies (i)–(iii), while a total order (or linear order) satisfies (i)–(iv).

Having defined the concept of a preorder, we consider an index set $\mathcal{I}$ and a subset $\widehat{\mathcal{I}} \subset \mathcal{I}$ which is CUD in the following, generalized sense:

Definition D.2. The subset $\widehat{\mathcal{I}} \subset \mathcal{I}$ is CUD if $i \leq j$ implies $i \in \widehat{\mathcal{I}}$ for all $i \in \mathcal{I}$ and $j \in \widehat{\mathcal{I}}$. $\widehat{\mathcal{I}}$ is also said to be downward closed with respect to $\leq$.

We also need a generalized notion of triangularity [see 18, Definition 1]:

Definition D.3. Let $\mathcal{I}$ be an index set equipped with a preorder, $\leq$. In addition, let $\mathbf{A}$ be a square matrix of size $|\mathcal{I}|$. We define $\leq$ -triangularity in the following way:

- (i) $\mathbf{A}$ is called lower $\leq$ -triangular if $A_{ij} \neq 0$ implies $j \leq i$ for all $i, j \in \mathcal{I}$.
- (ii) $\mathbf{A}$ is called upper $\leq$ -triangular if $A_{ij} \neq 0$ implies $i \leq j$ for all $i, j \in \mathcal{I}$.

In the context of grids, it is natural to introduce 1D index sets $\mathcal{I}^{(d)}$ equipped with preorders $\leq_d$ (note that the 1D indices need not be integers). The corresponding multi-indices are defined as in the main text:

$$\mathcal{I} = \mathcal{I}^{(1)} \times \dots \times \mathcal{I}^{(D)}. \quad (\text{D1})$$

The 1D preorders in turn induce a preorder $\leq$ on $\mathcal{I}$ [see 18, Definition 2]:

Definition D.4. For multi-indices $\mathbf{i}, \mathbf{j} \in \mathcal{I}$, we define $\mathbf{i} \leq \mathbf{j}$ if and only if $i_d \leq_d j_d$ for all $1 \leq d \leq D$.

With these definitions in place, Theorems E1 and E2 hold as they stand [see 18, Theorems 1 and 3, for proofs and formal statements]. These theorems allow kernel MVPs to be computed without reference to the underlying complete grid, thus eliminating the curse of dimensionality. The concrete computational complexity will, however, depend on the details of the grid.

One grid type of interest is the classical sparse grid [37], whose hierarchical structure makes it CUD. Sparse grids are discussed in detail by Holzmüller and Pflüger [18], so we will not consider them any further apart from mentioning their potential use in a GPR context.

In the main text we considered the 1D index sets

$$\mathcal{I}^{(m)} = \{0, 1, \dots, n_m - 1\}, \quad (\text{D2})$$

and we tacitly assumed a particular preorder, namely the standard order on the integers. A matrix that is triangular with respect to the standard order is simply triangular in the ordinary sense. It is, however, possible to choose a different preorder under which the MCR grid (Definitions C.1 and C.2) is still CUD, i.e. a different preorder that is consistent with the MCR framework. The key observation is that the non-zero indices $a_m \in \{1, 2, \dots, n_m - 1\} = \mathcal{A}^{(m)}$ are all treated on an equal footing. Only the reference index is special. This suggests the following preorder on $\mathcal{I}^{(m)}$:

Example D.5. Let $\mathcal{I}^{(m)} = \{0, 1, \dots, n-1\}$ and $\mathcal{A}^{(m)} = \{1, 2, \dots, n-1\}$. Let $\leq_m$ be a binary relation on $\mathcal{I}^{(m)}$ defined by:

- (i) $0 \leq_m 0$.
- (ii) $0 \leq_m a$ for all $a \in \mathcal{A}^{(m)}$.
- (iii) $a \leq_m b$ for all $a, b \in \mathcal{A}^{(m)}$.

$\leq_m$ is easily verified to be a preorder. Let $\bar{n} = n-1$. The following square matrices of size n are lower and upper triangular, respectively, with respect to $\leq_m$:

$$\mathbf{L}^{(m)} = \left[\begin{array}{c|cccc} l_{0,0} & 0 & 0 & \cdots & 0 \\ \hline l_{1,0} & l_{1,1} & l_{1,2} & \cdots & l_{1,\bar{n}} \\ l_{2,0} & l_{2,1} & l_{2,2} & \cdots & l_{2,\bar{n}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{\bar{n},0} & l_{\bar{n},1} & l_{\bar{n},2} & \cdots & l_{\bar{n},\bar{n}} \end{array} \right], \quad \mathbf{U}^{(m)} = \left[\begin{array}{c|cccc} u_{0,0} & u_{0,1} & u_{0,2} & \cdots & u_{0,\bar{n}} \\ \hline 0 & u_{1,1} & u_{1,2} & \cdots & u_{1,\bar{n}} \\ 0 & u_{2,1} & u_{2,2} & \cdots & u_{2,\bar{n}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & u_{\bar{n},1} & u_{\bar{n},2} & \cdots & u_{\bar{n},\bar{n}} \end{array} \right]. \quad (\text{D3})$$

E Chopped Kronecker products

The following theorems provide the mathematical underpinning that enables our work:

Theorem E1. *Let the index set $\widehat{\mathcal{I}}$ be CUD and let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{N \times N}$. We then have:*

- (i) *If $\mathbf{A}$ is lower triangular or $\mathbf{B}$ is upper triangular, then $\widehat{\mathbf{A}}\widehat{\mathbf{B}} = \widehat{\mathbf{A}}\widehat{\mathbf{B}}$.*
- (ii) *If $\mathbf{A}$ is lower (upper) triangular and invertible, then $\mathbf{A}^{-1}$ is lower (upper) triangular and $\widehat{\mathbf{A}}$ is invertible with $\widehat{\mathbf{A}}^{-1} = \widehat{\mathbf{A}}^{-1}$.*

Proof. See Holzmüller and Pflüger [18, Theorem 1]. □

Theorem E2. *Define the notation*

$$(j_m | \mathbf{i}_{-m}) = (i_1, \dots, i_{m-1}, j_m, i_{m+1}, \dots, i_D),$$

$$\widehat{\mathcal{I}}(\mathbf{i}, m) = \{j_m \in \mathcal{I}^{(m)} \mid (j_m | \mathbf{i}_{-m}) \in \widehat{\mathcal{I}}\}.$$

Using this notation, a matrix-vector product of the form $\widehat{\mathbf{w}} = \widehat{\mathbf{A}}^{[m]} \widehat{\mathbf{v}}$ can be computed as

$$\widehat{\mathbf{w}}_{\mathbf{i}} = \sum_{j_m \in \widehat{\mathcal{I}}(\mathbf{i}, m)} A_{i_m j_m}^{(m)} v_{(j_m | \mathbf{i}_{-m})}, \quad \mathbf{i} \in \widehat{\mathcal{I}}.$$

The complexity is at most $\mathcal{O}(|\mathcal{I}^{(m)}| \cdot |\widehat{\mathcal{I}}|)$.

Proof. See Holzmüller and Pflüger [18, Theorem 3]. □

Corollary E3. *Let $\{\mathbf{A}^{(m)}\}_{m=1}^D$ be square matrices of size n_m with LU decompositions $\mathbf{A}^{(m)} = \mathbf{L}^{(m)} \mathbf{U}^{(m)}$. For a fixed $k \in \{1, 2, \dots, D\}$, let $\mathbf{M}^{(k)} \in \mathbb{R}^{n_k \times n_k}$ be a square matrix that does not possess an LU decomposition. We define*

$$\mathbf{A} = \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(k)} \otimes \dots \otimes \mathbf{A}^{(D)} = \left(\prod_m \mathbf{L}^{[m]} \right) \left(\prod_m \mathbf{U}^{[m]} \right),$$

$$\mathbf{A}' = \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{M}^{(k)} \otimes \dots \otimes \mathbf{A}^{(D)} = \left(\prod_{m \neq k} \mathbf{L}^{[m]} \right) \mathbf{M}^{[k]} \left(\prod_{m \neq k} \mathbf{U}^{[m]} \right).$$

As a direct consequence of Theorem E1, we have

$$\widehat{\mathbf{A}} = \left(\prod_m \widehat{\mathbf{L}}^{[m]} \right) \left(\prod_m \widehat{\mathbf{U}}^{[m]} \right), \quad \widehat{\mathbf{A}'} = \left(\prod_{m \neq k} \widehat{\mathbf{L}}^{[m]} \right) \widehat{\mathbf{M}}^{[k]} \left(\prod_{m \neq k} \widehat{\mathbf{U}}^{[m]} \right).$$

The chopped Kronecker products $\widehat{\mathbf{A}}$ and $\widehat{\mathbf{A}'}$ can be multiplied with a vector by repeated application of Theorem E2. The complexity is at most $\mathcal{O}((n_1 + \dots + n_D) \cdot |\widehat{\mathcal{I}}|)$.

The details of how to implement Theorem E2 depend on the precise nature of $\widehat{\mathcal{I}}$. In our case, an algorithm can be formulated entirely in terms of the subtensors $\mathbf{v}^{(\mathbf{m})}$ and $\mathbf{w}^{(\mathbf{m})}$ rather than the individual elements of $\widehat{\mathbf{v}}$ and $\widehat{\mathbf{w}}$ (see Appendix F). This also means that the scalar multiplication in Theorem E2 is replaced by standard operations on the subtensors (scaling, outer products and one-index contractions), which is a great advantage in terms of performance.

For completeness, we note that Theorem E1(ii) allows direct inversion of a separable kernel defined over an incomplete grid. To see this, consider the complete-grid kernel in (7) and introduce the LU (Cholesky) decomposition of the base kernels. We then have

$$\mathbf{K} = \mathbf{K}^{[1]} \mathbf{K}^{[2]} \dots \mathbf{K}^{[D]} = (\mathbf{L}^{[1]} \mathbf{L}^{[2]} \dots \mathbf{L}^{[D]}) (\mathbf{U}^{[1]} \mathbf{U}^{[2]} \dots \mathbf{U}^{[D]}). \quad (\text{E1})$$

By applying Theorem E1, we find

$$\widehat{\mathbf{K}} = (\widehat{\mathbf{L}}^{[1]} \widehat{\mathbf{L}}^{[2]} \dots \widehat{\mathbf{L}}^{[D]}) (\widehat{\mathbf{U}}^{[1]} \widehat{\mathbf{U}}^{[2]} \dots \widehat{\mathbf{U}}^{[D]}), \quad (\text{E2})$$

$$\widehat{\mathbf{K}}^{-1} = (\widehat{\mathbf{U}}^{[1]} \widehat{\mathbf{U}}^{[2]} \dots \widehat{\mathbf{U}}^{[D]}) (\widehat{\mathbf{L}}^{[1]} \widehat{\mathbf{L}}^{[2]} \dots \widehat{\mathbf{L}}^{[D]}) \quad (\text{E3})$$

where we have temporarily used an overbar to denote the inverse to avoid clutter. For example,

$$\bar{\mathbf{U}}^{[m]} = (\mathbf{U}^{[m]})^{-1} = \mathbf{I}^{(1)} \otimes \dots \otimes (\mathbf{U}^{(m)})^{-1} \otimes \dots \otimes \mathbf{I}^{(D)}. \quad (\text{E4})$$

Unfortunately, this format in (E2) does not allow addition of noise, in contrast to the complete-grid method of Ishida et al. [5], which is based on eigendecomposition rather than LU decomposition. Even in applications where the data is essentially noiseless, noise is usually required to regularize the inverse. Although we are not willing to rule out this approach completely, our initial investigations were not promising due to very ill-conditioned numerics. In addition, the MVP complexity associated with (E2) and (E3) is $\mathcal{O}(nD\hat{N})$ compared to $\mathcal{O}(n\alpha\hat{N})$, which is achieved by CUTS-GPR. When $\alpha \ll D$, this difference is very significant. Finally, we note that the additive kernel used in CUTS-GPR is often more attractive than a separable kernel, especially in high dimension.

F Contractions with an MCR tensor

The aim of this appendix is to specialize Theorem E2 to our MCR-based framework. Theorem E2 suggests an algorithm with an outer loop over the elements of the result vector, $\widehat{\mathbf{w}}$. It is instructive, however, to start by considering a simplistic algorithm where the outer loop runs over the input vector, $\widehat{\mathbf{v}}$, instead:

Algorithm 1 Compute $\widehat{\mathbf{w}} = \widehat{\mathbf{A}}^{[m]} \widehat{\mathbf{v}}$ using Theorem E2.

```

for  $\mathbf{i} \in \widehat{\mathcal{I}}$  do
  for  $j_m \in \widehat{\mathcal{I}}^{(m)}$  do
    if  $(j_m | \mathbf{i}_{-m}) \in \widehat{\mathcal{I}}$  then
       $w_{(j_m | \mathbf{i}_{-m})} += A_{j_m i_m}^{(m)} v_{(i_m | \mathbf{i}_{-m})}$ 

```

Our goal is not to state all the specifics of our concrete implementation, but rather to explain the intuitions of how the algorithm works and how it may be implemented using standard tensor operations. The final algorithm is well-known in vibrational structure theory, where it is derived using arguments from quantum mechanics [38, 39].

It is important to note how the matrix $\mathbf{A}^{(m)}$ connects the input multi-index $\mathbf{i} = (i_m | \mathbf{i}_{-m})$ to the result multi-index $\mathbf{i}' = (j_m | \mathbf{i}_{-m})$. In particular, $\mathbf{A}^{(m)}$ acts to make the substitution $i_m \rightarrow j_m$. We will examine how this substitution affects a multi-index $\mathbf{i} \in \widehat{\mathcal{I}}$ and, in particular, under which circumstances an out-of-space multi-index $\mathbf{i}' \notin \widehat{\mathcal{I}}$ is produced.

We start by recalling a few definitions. Let $\mathcal{M}_{\text{grid}}$ be an MCR that is CUTS (Definition 3.2) and let

$$\widehat{\mathcal{I}} = \bigcup_{\mathbf{m} \in \mathcal{M}_{\text{grid}}} \mathcal{I}^{(\mathbf{m})} \quad (\text{F1})$$

be the corresponding incomplete grid (Definitions C.1 and C.2). For a multi-index $\mathbf{i} \in \mathcal{I}^{(\mathbf{m})}$ we make the association $\mathbf{i} = (i_1, \dots, i_D) \cong (\mathbf{m}, \mathbf{a})$ (see Definition C.1) such that for all $m = 1, \dots, D$

$$i_m = a_m > 0 \quad \text{if } m \in \mathbf{m}, \quad (\text{F2a})$$

$$i_m = 0 \quad \text{if } m \notin \mathbf{m}. \quad (\text{F2b})$$

In other words, $\mathbf{m}$ indicates the positions of the non-zero indices, which are collected in $\mathbf{a}$. We use the notation

$$\mathbf{i}_{-m} = (i_1, \dots, i_{m-1}, i_{m+1}, \dots, i_D), \quad (\text{F3})$$

$$(j_m | \mathbf{i}_{-m}) = (i_1, \dots, i_{m-1}, j_m, i_{m+1}, \dots, i_D) \quad (\text{F4})$$

to mean the multi-indices obtained from $\mathbf{i}$ by removing i_m and by making the replacement $i_m \rightarrow j_m$, respectively. Now consider a fixed $\mathbf{m} \in \mathcal{M}_{\text{grid}}$ and a multi-index $\mathbf{i} \in \mathcal{I}^{(\mathbf{m})}$. We first choose a mode $m \in \mathbf{m}$ and replace the non-zero index $i_m = a_m > 0$ by either zero or by a non-zero index $b_m \in \mathcal{A}^{(m)}$. The following then holds:

$$(0 | \mathbf{i}_{-m}) \cong (\mathbf{m} \setminus m, \mathbf{a}_{-m}) \in \mathcal{I}^{(\mathbf{m} \setminus m)}, \quad (\text{F5})$$

$$(b_m | \mathbf{i}_{-m}) \cong (\mathbf{m}, (b_m | \mathbf{a}_{-m})) \in \mathcal{I}^{(\mathbf{m})}. \quad (\text{F6})$$

If instead we choose $m \notin \mathbf{m}$ and replace the index $i_m = 0$, we find that

$$(0 | \mathbf{i}_{-m}) \cong (\mathbf{m}, \mathbf{a}) \in \mathcal{I}^{(\mathbf{m})}, \quad (\text{F7})$$

$$(b_m | \mathbf{i}_{-m}) \cong (\mathbf{m} \cup m, (b_m | \mathbf{a})) \in \mathcal{I}^{(\mathbf{m} \cup m)}. \quad (\text{F8})$$

This can be summarized as

$$\begin{aligned}
 \text{if } m \in \mathbf{m} & : \begin{cases} \mathbf{i} \in \mathcal{I}^{(\mathbf{m})} \xrightarrow{a_m \rightarrow 0} \mathbf{i}' \in \mathcal{I}^{(\mathbf{m} \setminus m)} & (\text{down}) \\ \mathbf{i} \in \mathcal{I}^{(\mathbf{m})} \xrightarrow{a_m \rightarrow b_m} \mathbf{i}' \in \mathcal{I}^{(\mathbf{m})} & (\text{forward}) \end{cases} \\
 \text{if } m \notin \mathbf{m} & : \begin{cases} \mathbf{i} \in \mathcal{I}^{(\mathbf{m})} \xrightarrow{0 \rightarrow 0} \mathbf{i} \in \mathcal{I}^{(\mathbf{m})} & (\text{passive}) \\ \mathbf{i} \in \mathcal{I}^{(\mathbf{m})} \xrightarrow{0 \rightarrow b_m} \mathbf{i}' \in \mathcal{I}^{(\mathbf{m} \cup m)} & (\text{up}) \end{cases}
 \end{aligned}$$

Since $\mathcal{M}_{\text{grid}}$ it CUTS, we know that $\mathbf{m} \setminus m \in \mathcal{M}_{\text{grid}}$ and $\mathcal{I}^{(\mathbf{m} \setminus m)} \subset \widehat{\mathcal{I}}$. It may, however, be the case that $\mathbf{m} \cup m \notin \mathcal{M}_{\text{grid}}$, so the last substitution may produce an out-of-space multi-index $\mathbf{i}' \notin \widehat{\mathcal{I}}$. The four classes of substitutions are called *down*, *forward*, *passive* and *up*.

Now let $\widehat{\mathbf{v}}$ be a vector or tensor indexed over $\widehat{\mathcal{I}}$. With the MCR construction in mind, it is natural to view $\widehat{\mathbf{v}}$ as a collection of subtensors $\mathbf{v}^{(\mathbf{m})}$, just like $\widehat{\mathcal{I}}$ is a collection of subgrids $\mathcal{I}^{(\mathbf{m})}$. The subtensors have elements $\mathbf{v}_{\mathbf{a}}^{(\mathbf{m})}$ for all $\mathbf{a} \in \mathcal{A}^{(\mathbf{m})}$. Before introducing the subtensor notation into the algorithm, we simply split the main body of the algorithm according to the four types of index substitutions:

Algorithm 2 Compute $\widehat{\mathbf{w}} = \widehat{\mathbf{A}^{[m]}} \widehat{\mathbf{v}}$ using Theorem E2.

```

for  $\mathbf{i} \in \widehat{\mathcal{I}}$  do
  for  $j_m \in \mathcal{I}^{(m)}$  do
    if  $j_m = 0$  and  $i_m > 0$  then                                ▷ down
       $w_{(0|\mathbf{i}_{-m})} += A_{0i_m}^{(m)} v_{\mathbf{i}}$ 
    else if  $j_m > 0$  and  $i_m > 0$  then                            ▷ forward
       $w_{(j_m|\mathbf{i}_{-m})} += A_{j_m i_m}^{(m)} v_{\mathbf{i}}$ 
    else if  $j_m = 0$  and  $i_m = 0$  then                            ▷ passive
       $w_{(0|\mathbf{i}_{-m})} += A_{00}^{(m)} v_{\mathbf{i}}$ 
    else if  $j_m > 0$  and  $i_m = 0$  then                            ▷ up
      if  $(j_m|\mathbf{i}_{-m}) \in \widehat{\mathcal{I}}$  then                                    ▷ skip out-of-space
         $w_{(j_m|\mathbf{i}_{-m})} += A_{j_m 0}^{(m)} v_{\mathbf{i}}$ 

```

This rewrite is of course valid for any choice of $\widehat{\mathcal{I}}$, but it is particularly well-suited for the MCR-based construction. We now replace the loop over $\mathbf{i} \in \widehat{\mathcal{I}}$ with an equivalent nested loop over $\mathbf{m} \in \mathcal{M}_{\text{grid}}$ and $\mathbf{a} \in \mathcal{A}^{(\mathbf{m})}$. In addition, we observe that $i_m = 0$ implies to $m \in \mathbf{m}$, while $i_m > 0$ implies to $m \notin \mathbf{m}$. Next, we replace the loop over $j_m \in \mathcal{I}^{(m)}$ with a separate line covering $j_m = 0$ and a loop over $b_m \in \mathcal{A}^{(m)} = \mathcal{I}^{(m)} \setminus \{0\}$. Finally, we introduce the subtensor notation to obtain the following algorithm:

Algorithm 3 Compute $\widehat{\mathbf{w}} = \widehat{\mathbf{A}^{[m]}} \widehat{\mathbf{v}}$ using Theorem E2.

```

for  $\mathbf{m} \in \mathcal{M}_{\text{grid}}$  do
  for  $\mathbf{a} \in \mathcal{A}^{(\mathbf{m})}$  do
    if  $m \in \mathbf{m}$  then
       $w_{\mathbf{a}_{-m}}^{(\mathbf{m} \setminus m)} += A_{0a_m}^{(m)} v_{\mathbf{a}}^{(\mathbf{m})}$                                 ▷ down
      for  $b_m \in \mathcal{A}^{(m)}$  do
         $w_{(b_m|\mathbf{a}_{-m})}^{(\mathbf{m})} += A_{b_m a_m}^{(m)} v_{\mathbf{a}}^{(\mathbf{m})}$                             ▷ forward
      else
         $w_{\mathbf{a}}^{(\mathbf{m})} += A_{00}^{(m)} v_{\mathbf{a}}^{(\mathbf{m})}$                                 ▷ passive
        if  $(\mathbf{m} \cup m) \in \mathcal{M}_{\text{grid}}$  then                                ▷ skip out-of-space
          for  $b_m \in \mathcal{A}^{(m)}$  do
             $w_{(b_m|\mathbf{a})}^{(\mathbf{m} \cup m)} += A_{b_m 0}^{(m)} v_{\mathbf{a}}^{(\mathbf{m})}$                                 ▷ up

```

Algorithm 3 suggests the following partitioning of the one-mode matrix (see also Example D.5):

$$\mathbf{A}^{(m)} = \left[\begin{array}{c|c} p^{(m)} & \mathbf{d}^{(m)} \\ \hline \mathbf{u}^{(m)} & \mathbf{F}^{(m)} \end{array} \right]. \quad (\text{F9})$$

The vectors $\mathbf{d}^{(m)}$ and $\mathbf{u}^{(m)}$ contain the *down* and *up* elements ($A_{0b_m}^{(m)}$ and $A_{a_m0}^{(m)}$), respectively, while the matrix $\mathbf{F}^{(m)}$ contains the *forward* elements ($A_{a_m b_m}^{(m)}$). The passive element is $p^{(m)} = A_{00}^{(m)}$. With this notation we see that

- (i) The *down* contraction is a one-index contraction between the vector $\mathbf{d}^{(m)}$ and the tensor $\mathbf{v}^{(\mathbf{m})}$.
- (ii) The *forward* contraction is a one-index contraction between the matrix $\mathbf{F}^{(m)}$ and the tensor $\mathbf{v}^{(\mathbf{m})}$.
- (iii) The *passive* contraction is a scalar multiplication between the scalar $p^{(m)}$ and the tensor $\mathbf{v}^{(\mathbf{m})}$.
- (iv) The *up* contraction is an outer product between the vector $\mathbf{u}^{(m)}$ and the tensor $\mathbf{v}^{(\mathbf{m})}$.

A one-index contraction in mode m is also known as an m -mode product [16]. Assuming $n_1 = \dots = n_D = n$ for simplicity and letting $k = |\mathbf{m}|$, the complexities are as follows:

- (i) Down: $\mathcal{O}(n^k)$
- (ii) Forward: $\mathcal{O}(n^{k+1})$
- (iii) Passive: $\mathcal{O}(n^k)$
- (iv) Up: $\mathcal{O}(n^{k+1})$

Since the contraction index m is clear from context, we simply use the symbol $\otimes$ to denote the vector-tensor outer product and the symbol $\times$ to denote the m -mode product. With this notation, we can restate Algorithm 3 as Algorithm 4, which is generic in the sense that all elements of $\mathbf{A}^{(m)}$ and $\hat{\mathbf{v}}$ may be non-zero. However, it is obvious that the algorithm can be simplified if certain blocks of $\mathbf{A}^{(m)}$ are zero (Appendix G.2) or if certain subtensors in $\hat{\mathbf{v}}$ are zero (Appendix G.3).

Algorithm 4 Compute $\hat{\mathbf{w}} = \widehat{\mathbf{A}^{[m]}} \hat{\mathbf{v}}$ using Theorem E2.

```

for  $\mathbf{m} \in \mathcal{M}_{\text{grid}}$  do
  if  $m \in \mathbf{m}$  then
     $\mathbf{w}^{(\mathbf{m} \setminus m)} \mathrel{+}= \mathbf{d}^{(m)} \times \mathbf{v}^{(\mathbf{m})}$   $\triangleright$  down
     $\mathbf{w}^{(\mathbf{m})} \mathrel{+}= \mathbf{F}^{(m)} \times \mathbf{v}^{(\mathbf{m})}$   $\triangleright$  forward
  else
     $\mathbf{w}^{(\mathbf{m})} \mathrel{+}= p^{(m)} \mathbf{v}^{(\mathbf{m})}$   $\triangleright$  passive
    if  $(\mathbf{m} \cup m) \in \mathcal{M}_{\text{grid}}$  then  $\triangleright$  skip out-of-space
       $\mathbf{w}^{(\mathbf{m} \cup m)} \mathrel{+}= \mathbf{u}^{(m)} \otimes \mathbf{v}^{(\mathbf{m})}$   $\triangleright$  up

```

G Kernel matrix-vector products

G.1 Deriving the kernel MVP

Using (16) and the properties of the bracket matrices, we rewrite (14) as

$$\begin{aligned}
\mathbf{K} &= \sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \mathbf{K}^{[\mathbf{m}]} \mathbf{J}^{[-\mathbf{m}]} \\
&= \sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \mathbf{K}^{[\mathbf{m}]} \mathbf{L}^{[-\mathbf{m}]} \mathbf{R}^{[-\mathbf{m}]} \mathbf{U}^{[-\mathbf{m}]} \\
&= \sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \mathbf{L}^{[-\mathbf{m}]} \left[\mathbf{L}^{[\mathbf{m}]} (\mathbf{L}^{[\mathbf{m}]})^{-1} \right] \mathbf{K}^{[\mathbf{m}]} \mathbf{R}^{[-\mathbf{m}]} \left[(\mathbf{U}^{[\mathbf{m}]})^{-1} \mathbf{U}^{[\mathbf{m}]} \right] \mathbf{U}^{[-\mathbf{m}]} \\
&= \mathbf{L} \left[\sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 (\mathbf{L}^{[\mathbf{m}]})^{-1} \mathbf{K}^{[\mathbf{m}]} (\mathbf{U}^{[\mathbf{m}]})^{-1} \mathbf{R}^{[-\mathbf{m}]} \right] \mathbf{U} \\
&= \mathbf{L} \left[\sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \mathbf{M}^{[\mathbf{m}]} \mathbf{R}^{[-\mathbf{m}]} \right] \mathbf{U} \\
&= \mathbf{L} \mathbf{M} \mathbf{U}.
\end{aligned} \tag{G1}$$

At the second equality we use the factorization of $\mathbf{J}^{(m)}$ (16), at the third equality we insert identities (in square brackets), and at the fourth equality we observe

$$\mathbf{L}^{[-\mathbf{m}]} \mathbf{L}^{[\mathbf{m}]} = \mathbf{L}^{[1]} \dots \mathbf{L}^{[D]} = \mathbf{L}^{(1)} \otimes \dots \otimes \mathbf{L}^{(D)} \equiv \mathbf{L}, \tag{G2}$$

$$\mathbf{U}^{[\mathbf{m}]} \mathbf{U}^{[-\mathbf{m}]} = \mathbf{U}^{[1]} \dots \mathbf{U}^{[D]} = \mathbf{U}^{(1)} \otimes \dots \otimes \mathbf{U}^{(D)} \equiv \mathbf{U}. \tag{G3}$$

At the last two lines of (18), we simply make the definitions

$$\mathbf{M}^{(m)} = (\mathbf{L}^{(m)})^{-1} \mathbf{K}^{(m)} (\mathbf{U}^{(m)})^{-1}, \tag{G4}$$

$$\mathbf{M} = \left[\sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \mathbf{M}^{[\mathbf{m}]} \mathbf{R}^{[-\mathbf{m}]} \right]. \tag{G5}$$

G.2 Complexity of MVPs with $\widehat{\mathbf{L}}$ and $\widehat{\mathbf{U}}$

In order to determine the complexity of the MVP

$$\widehat{\mathbf{U}} \widehat{\mathbf{v}} = (\widehat{\mathbf{U}}^{[1]} \widehat{\mathbf{U}}^{[1]} \dots \widehat{\mathbf{U}}^{[D]}) \widehat{\mathbf{v}} \tag{G6}$$

we start by considering a single one-mode contraction $\widehat{\mathbf{U}}^{[m]} \widehat{\mathbf{v}}$. The underlying 1D matrix $\mathbf{U}^m$ can be decomposed as

$$\mathbf{U}^m = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & \dots & 1 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix} = \mathbf{I}^{(m)} + \mathbf{S}^{(m)} \tag{G7}$$

which implies

$$\widehat{\mathbf{U}}^{[m]} \widehat{\mathbf{v}} = (\widehat{\mathbf{I}}^{[m]} + \widehat{\mathbf{S}}^{[m]}) \widehat{\mathbf{v}} = \widehat{\mathbf{v}} + \widehat{\mathbf{S}}^{[m]} \widehat{\mathbf{v}}. \tag{G8}$$

The only non-zero block of the matrix $\mathbf{S}^{(m)}$ is the down-block, so the second term in (G8) only involves down-contractions in mode m . This means that only MCs containing m are touched.

For definiteness, we consider an MCR, $\mathcal{M}_{\text{grid}}$, defined by a maximum cut level, α . The MCR is dominated by the α -cuts, which will thus determine the complexity. The number of α -cuts containing the given mode m is

$$\binom{D-1}{\alpha-1} \sim \mathcal{O}\left(\frac{(D-1)^{\alpha-1}}{(\alpha-1)!}\right) = \mathcal{O}\left(\frac{D^{\alpha-1}}{(\alpha-1)!}\right). \tag{G9}$$

Assuming $n_1 = \dots = n_D = n$ for simplicity, each down-contraction has an operation count of n^α , so the complexity of $\widehat{\mathbf{U}}^{[m]}\widehat{\mathbf{v}}$ is

$$\mathcal{O}\left(\frac{D^{\alpha-1}n^\alpha}{(\alpha-1)!}\right). \quad (\text{G10})$$

Equation (G6) involves D MVPs with this complexity, so the overall cost of $\widehat{\mathbf{U}}\widehat{\mathbf{v}}$ is

$$\mathcal{O}\left(\frac{D^\alpha n^\alpha}{(\alpha-1)!}\right) = \mathcal{O}(\alpha\widehat{N}). \quad (\text{G11})$$

The complexity of $\widehat{\mathbf{L}}\widehat{\mathbf{v}}$ can be established in much the same way. To start with, we note that

$$\mathbf{L}^m = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 1 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 0 & \dots & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & \dots & 0 \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 0 & \dots & 0 \end{bmatrix} = \mathbf{I}^{(m)} + \mathbf{D}^{(m)} \quad (\text{G12})$$

which implies

$$\widehat{\mathbf{L}}^{[m]}\widehat{\mathbf{v}} = (\widehat{\mathbf{I}}^{[m]} + \widehat{\mathbf{D}}^{[m]})\widehat{\mathbf{v}} = \widehat{\mathbf{v}} + \widehat{\mathbf{D}}^{[m]}\widehat{\mathbf{v}}. \quad (\text{G13})$$

This time, the second term involves up-contractions in mode m , so only MCs excluding m are touched. Due to the chopping we only need to consider those up-contractions that remain in $\mathcal{M}_{\text{grid}}$. Doing an up-contraction on an α -cut will for example produce a result of order $\alpha + 1$, which is outside $\mathcal{M}_{\text{grid}}$. The dominating up-contractions are thus those that touch MCs of order $\alpha - 1$ that do not contain the given mode m . The number of such MCs is

$$\binom{D-1}{\alpha-1} \sim \mathcal{O}\left(\frac{(D-1)^{\alpha-1}}{(\alpha-1)!}\right) = \mathcal{O}\left(\frac{D^{\alpha-1}}{(\alpha-1)!}\right) \quad (\text{G14})$$

while the cost of each up-contraction is n^α . In conclusion, we find that $\widehat{\mathbf{L}}\widehat{\mathbf{v}}$ has the same complexity as $\widehat{\mathbf{U}}\widehat{\mathbf{v}}$, namely $\mathcal{O}(\alpha\widehat{N})$.

G.3 Complexity of MVPs with $\widehat{\mathbf{M}}$

We will now address the complexity of the MVP with the matrix

$$\widehat{\mathbf{M}} = \sum_{\mathbf{m}} \sigma_{|\mathbf{m}|}^2 \widehat{\mathbf{M}}^{[\mathbf{m}]} \widehat{\mathbf{R}}^{[\neg\mathbf{m}]}. \quad (\text{G15})$$

For simplicity, we will start by considering $\widehat{\mathbf{R}}^{[\neg\mathbf{m}]}\widehat{\mathbf{v}}$ and then $\widehat{\mathbf{M}}^{[\mathbf{m}]} \widehat{\mathbf{R}}^{[\neg\mathbf{m}]}\widehat{\mathbf{v}}$ before summing up the terms. The key component is the matrix $\widehat{\mathbf{R}}^{[\neg\mathbf{m}]}$, which projects onto the subspace where all modes $m \notin \mathbf{m}$ are in the reference (equivalently, this is the subspace where only the modes $m \in \mathbf{m}$ are allowed to be displaced from the reference). This subspace can generally be characterized by a set of multi-indices $\widehat{\mathcal{I}}(\mathbf{m}) \subseteq \widehat{\mathcal{I}}$, but for our purposes it is more useful to consider the corresponding MCR, $\mathcal{M}(\mathbf{m}) \subseteq \mathcal{M}_{\text{grid}}$. Due to the action of the projector (as described above), we must have

$$\mathcal{M}(\mathbf{m}) = \{\mathbf{m}' \mid \mathbf{m}' \subseteq \mathbf{m}, \mathbf{m}' \in \mathcal{M}_{\text{grid}}\}. \quad (\text{G16})$$

In our implementation we currently assume $\mathcal{M}_{\text{kernel}} \subseteq \mathcal{M}_{\text{grid}}$, so we always have $\mathbf{m} \in \mathcal{M}_{\text{grid}}$. But $\mathcal{M}_{\text{grid}}$ is CUTS, so in this case (G16) simplifies to

$$\mathcal{M}(\mathbf{m}) = \{\mathbf{m}' \mid \mathbf{m}' \subseteq \mathbf{m}\} = \text{powerset}(\mathbf{m}), \quad (\text{G17})$$

which has size $|\mathcal{M}(\mathbf{m})| = 2^{|\mathbf{m}|}$.

In addition to being a projector, $\widehat{\mathbf{R}}^{[\neg\mathbf{m}]}$ commutes with $\widehat{\mathbf{M}}^{[\mathbf{m}]}$ so

$$\widehat{\mathbf{M}}^{[\mathbf{m}]} \widehat{\mathbf{R}}^{[\neg\mathbf{m}]} = \widehat{\mathbf{R}}^{[\neg\mathbf{m}]} \widehat{\mathbf{M}}^{[\mathbf{m}]} \widehat{\mathbf{R}}^{[\neg\mathbf{m}]}, \quad (\text{G18})$$

which shows that the action of $\widehat{\mathbf{M}}^{[\mathbf{m}]}$ does not produce any terms outside the subspace $\mathcal{M}(\mathbf{m})$. We conclude that each term in $\widehat{\mathbf{M}}$ touches only a small part of the vector, which is in fact the key to the performance of the MVP. The actual multiplication by $\widehat{\mathbf{M}}^{[\mathbf{m}]}$ can be carried out using the LU-based machinery (Theorem E1 and Corollary E3) with the only modification that the code should only access the sub-vector defined by $\mathcal{M}(\mathbf{m})$, rather than the full vector. If $\mathbf{m} \in \mathcal{M}_{\text{grid}}$ (as in our current implementation), the subspace $\mathcal{M}(\mathbf{m})$ can be viewed as a complete grid over the modes $\mathbf{m}$. This means that $\widehat{\mathbf{M}}^{[\mathbf{m}]}$ can be carried out as a sequence of ordinary one-mode contractions (no LU decomposition is necessary).

To derive a definite complexity, we consider identical kernel and grid MCRs ($\mathcal{M}_{\text{kernel}} = \mathcal{M}_{\text{grid}}$) defined by a maximum interaction order or cut level, α . The MVP is dominated by the highest-order kernel terms (the α -terms), and each such term is dominated by the forward contractions. Consider an α -term indexed by $\mathbf{m}$. Every non-empty subspace MC, $\mathbf{m}' \in \mathcal{M}(\mathbf{m})$, can be forward-contracted $k = |\mathbf{m}'|$ times, and each of those forward contraction has an operation count of $(n-1)^{k+1}$ (assuming $n_1 = \dots = n_D = n$). For a single α -term, this yields an operation count of

$$\sum_{k=1}^{\alpha} \binom{\alpha}{k} k(n-1)^{k+1} = \alpha n^{\alpha-1} (n-1)^2 < \alpha n^{\alpha+1}. \quad (\text{G19})$$

The first equality can be proved by differentiating the binomial theorem, $n^\alpha = \sum_{k=0}^{\alpha} \binom{\alpha}{k} (n-1)^k$, with respect to n and multiplying the result by $(n-1)^2$ on both sides. The number of α -terms is

$$\binom{D}{\alpha} \sim \mathcal{O}\left(\frac{D^\alpha}{\alpha!}\right), \quad (\text{G20})$$

so the overall complexity of the MVP by $\widehat{\mathbf{M}}$ is

$$\mathcal{O}\left(n\alpha \frac{D^\alpha n^\alpha}{\alpha!}\right) = \mathcal{O}(n\alpha \widehat{N}). \quad (\text{G21})$$

G.4 Complexity of MVPs with $\partial\widehat{\mathbf{M}}/\partial\ell^{(m)}$

The length scale derivative,

$$\frac{\partial\widehat{\mathbf{M}}}{\partial\ell^{(m)}} = \sum_{\substack{\mathbf{m} \in \mathcal{M}_{\text{kernel}}, \\ m \in \mathbf{m}}} \sigma_{|\mathbf{m}|}^2 \frac{\partial\widehat{\mathbf{M}}^{[\mathbf{m}]}}{\partial\ell^{(m)}} \widehat{\mathbf{R}}^{[-\mathbf{m}]}, \quad (\text{G22})$$

has the same form as $\widehat{\mathbf{M}}$ itself, except that the sum is restricted. This means that each of the dominating α -terms in (G22) has the same cost as an α -term in $\widehat{\mathbf{M}}$, namely $\mathcal{O}(\alpha n^{\alpha+1})$ (see Appendix G.3). The number of α -terms in (G22) is exactly the number α -terms containing the given mode m :

$$\binom{D-1}{\alpha-1} < \binom{D}{\alpha-1} \sim \mathcal{O}\left(\frac{D^{\alpha-1}}{(\alpha-1)!}\right). \quad (\text{G23})$$

We thus find that the complexity of (G22) is

$$\mathcal{O}\left(\alpha n^{\alpha+1} \frac{D^{\alpha-1}}{(\alpha-1)!}\right) = \mathcal{O}\left(\frac{n\alpha^2}{D} \frac{D^\alpha n^\alpha}{\alpha!}\right) = \mathcal{O}\left(\frac{n\alpha^2}{D} \widehat{N}\right). \quad (\text{G24})$$

Each term in the derivative MVP populates the subspace defined by $\mathcal{M}(\mathbf{m})$ (see Appendix G.3). In total, all α -cuts containing m as well as all lower-order cuts are populated. Counting the dominating α -cuts and using (G23), we find that the number of non-zero elements of $\widehat{\mathbf{w}} = (\partial\widehat{\mathbf{M}}/\partial\ell^{(m)})\widehat{\mathbf{v}}$ goes like

$$\mathcal{O}\left(n^\alpha \frac{D^{\alpha-1}}{(\alpha-1)!}\right) = \mathcal{O}\left(\frac{\alpha}{D} \widehat{N}\right). \quad (\text{G25})$$

Appendix H shows that an MVP of the form $\widehat{\mathbf{M}}\widehat{\mathbf{U}}\widehat{\mathbf{e}}_i$ can be computed directly (without forming the intermediate vector $\widehat{\mathbf{U}}\widehat{\mathbf{e}}_i$) at a cost of $\mathcal{O}(\alpha\widehat{N})$. This should be compared to the generic MVP $\widehat{\mathbf{M}}\widehat{\mathbf{U}}\widehat{\mathbf{v}}$ with complexity $\mathcal{O}(n\alpha\widehat{N})$. The same reduction by a factor n applies to the derivative MVP, so $(\partial\widehat{\mathbf{M}}/\partial\ell^{(m)})\widehat{\mathbf{U}}\widehat{\mathbf{e}}_i$ can be obtained with complexity

$$\mathcal{O}\left(\frac{\alpha^2}{D} \widehat{N}\right). \quad (\text{G26})$$

G.5 Reducing complexity in the quadratic term

In order to maintain a D -complexity of $\mathcal{O}(D^\alpha)$, we will use the factorized form of $\widehat{\mathbf{K}}$ (20):

$$\boldsymbol{\alpha}^\top \frac{\partial \widehat{\mathbf{K}}}{\partial \ell^{(m)}} \boldsymbol{\alpha} = (\widehat{\mathbf{U}} \boldsymbol{\alpha})^\top \frac{\partial \widehat{\mathbf{M}}}{\partial \ell^{(m)}} (\widehat{\mathbf{U}} \boldsymbol{\alpha}). \quad (\text{G27})$$

Appendix G.4 shows that a derivative MVP like $\widehat{\mathbf{w}} = (\partial \widehat{\mathbf{M}} / \partial \ell^{(m)}) \widehat{\mathbf{v}}$ has a complexity of only $\mathcal{O}(n\alpha^2 \widehat{N}/D)$. In addition, only a small fraction of $\widehat{\mathbf{w}}$ is non-zero, leading to $\mathcal{O}(\alpha \widehat{N}/D)$ dot products with $\widehat{\mathbf{w}}$. Computing (G27) for all D length scales can thus be done at a total cost of $\mathcal{O}(n\alpha^2 \widehat{N}) \sim \mathcal{O}(D^\alpha)$.

The same approach can be used for the bilinear forms that occur in the stochastic estimate of $\text{tr}(\mathbf{C}^{-1} \partial \mathbf{C} / \partial \theta)$ and the deterministic computation of preconditioner trace terms; see (A18) and (J10), respectively.

H Computing a single column of $\widehat{\mathbf{K}}$

In order to construct the pivoted Cholesky preconditioner (see Appendix A.1), we must be able to access a single column of the kernel matrix. For a given multi-index $\mathbf{i} \in \widehat{\mathcal{I}}$, we can compute the corresponding column of $\widehat{\mathbf{K}}$ by doing an MVP with the standard basis vector $\widehat{\mathbf{e}}_{\mathbf{i}} \in \mathbb{R}^{\widehat{N}}$:

$$\widehat{\mathbf{k}}_{\mathbf{i}} = \widehat{\mathbf{K}}\widehat{\mathbf{e}}_{\mathbf{i}}. \quad (\text{H1})$$

As described in Section 5.1 this has a complexity of $\mathcal{O}(n\alpha\widehat{N})$ (see also Appendices G.2 and G.3). In the following we show that this can be improved to $\mathcal{O}(\alpha\widehat{N})$. We start by applying Theorem E1 to $\widehat{\mathbf{K}}$ after which we introduce explicit chopping matrices:

$$\widehat{\mathbf{k}}_{\mathbf{i}} = \widehat{\mathbf{L}}\widehat{\mathbf{M}}\widehat{\mathbf{U}}\widehat{\mathbf{e}}_{\mathbf{i}} = \widehat{\mathbf{L}}(\mathbf{\Gamma}^{\top}\mathbf{M}\mathbf{U}\mathbf{\Gamma})(\mathbf{\Gamma}^{\top}\mathbf{e}_{\mathbf{i}}) = \widehat{\mathbf{L}}\mathbf{\Gamma}^{\top}\mathbf{M}\mathbf{U}\mathbf{e}_{\mathbf{i}}. \quad (\text{H2})$$

The vector $\mathbf{e}_{\mathbf{i}}$ is contained in the subspace corresponding to $\widehat{\mathcal{I}}$ so $\mathbf{\Gamma}\mathbf{\Gamma}^{\top}\mathbf{e}_{\mathbf{i}} = \mathbf{e}_{\mathbf{i}}$ even though $\mathbf{\Gamma}\mathbf{\Gamma}^{\top}$ is generally not the identity. The first part of the MVP is now expressed without chopping, which is usually not desirable (indeed, the chopping framework was introduced to avoid exactly this). It is useful, however, in this particular case since $\mathbf{e}_{\mathbf{i}}$ can be viewed as a rank-one tensor. To be specific, we make the association $\mathbf{i} \cong (\mathbf{m}, \mathbf{a})$ (see Definition C.2) and write

$$\mathbf{e}_{\mathbf{i}} = \bigotimes_{m=1}^D \mathbf{v}^{(m)}, \quad \mathbf{v}^{(m)} = \begin{cases} \mathbf{r}^{(m)} & \text{if } m \notin \mathbf{m}, \\ \mathbf{a}^{(m)} & \text{if } m \in \mathbf{m}. \end{cases} \quad (\text{H3})$$

The vector $\mathbf{v}^{(m)} \in \mathbb{R}^{n_m}$ has element i_m equal to one and the remaining elements equal to zero. The symbols $\mathbf{r}^{(m)}$ and $\mathbf{a}^{(m)}$ only serve to visually separate the cases $m \notin \mathbf{m}$ ($i_m = 0$) and $m \in \mathbf{m}$ ($i_m = a_m > 0$). To further simplify notation we reorder the factors of the Kronecker product, which corresponds to a permutation of the elements of $\mathbf{e}_{\mathbf{i}}$. To compensate, we multiply by the inverse permutation, $\mathbf{P}$, and rewrite (H3) as

$$\mathbf{e}_{\mathbf{i}} = \mathbf{P} \left[\left(\bigotimes_{m \in \mathbf{m}} \mathbf{a}^{(m)} \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}} \mathbf{r}^{(m)} \right) \right]. \quad (\text{H4})$$

We stress that the only purpose of this reordering is to make the equations more readable. No permutation is necessary in the actual computation.

Before proceeding we observe that

$$\mathbf{U}^{(m)}\mathbf{r}^{(m)} = \mathbf{r}^{(m)}, \quad (\text{H5a})$$

$$\mathbf{U}^{(m)}\mathbf{a}^{(m)} = \mathbf{r}^{(m)} + \mathbf{a}^{(m)}, \quad (\text{H5b})$$

$$\mathbf{R}^{(m)}\mathbf{r}^{(m)} = \mathbf{r}^{(m)}, \quad (\text{H5c})$$

$$\mathbf{R}^{(m)}(\mathbf{r}^{(m)} + \mathbf{a}^{(m)}) = \mathbf{r}^{(m)}. \quad (\text{H5d})$$

Using (H5a) and (H5b), we find

$$\begin{aligned} \mathbf{U}\mathbf{e}_{\mathbf{i}} &= \left(\bigotimes_{m=1}^D \mathbf{U}^{(m)} \right) \mathbf{P} \left[\left(\bigotimes_{m \in \mathbf{m}} \mathbf{a}^{(m)} \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}} \mathbf{r}^{(m)} \right) \right] \\ &= \mathbf{P} \left[\left(\bigotimes_{m \in \mathbf{m}} \mathbf{U}^{(m)}\mathbf{a}^{(m)} \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}} \mathbf{U}^{(m)}\mathbf{r}^{(m)} \right) \right] \\ &= \mathbf{P} \left[\left(\bigotimes_{m \in \mathbf{m}} (\mathbf{r}^{(m)} + \mathbf{a}^{(m)}) \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}} \mathbf{r}^{(m)} \right) \right]. \end{aligned} \quad (\text{H6})$$

Substituting this into (H2) and applying (H5c) and (H5d) then yields

$$\begin{aligned}
\hat{\mathbf{k}}_i &= \hat{\mathbf{L}}\mathbf{\Gamma}^\top \mathbf{M}\mathbf{U}\mathbf{e}_i \\
&= \hat{\mathbf{L}}\mathbf{\Gamma}^\top \left(\sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \mathbf{M}^{[\mathbf{m}']} \mathbf{R}^{[-\mathbf{m}']} \mathbf{P} \left[\left(\bigotimes_{m \in \mathbf{m}} (\mathbf{r}^{(m)} + \mathbf{a}^{(m)}) \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}} \mathbf{r}^{(m)} \right) \right] \right) \\
&= \hat{\mathbf{L}}\mathbf{\Gamma}^\top \left(\sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \mathbf{M}^{[\mathbf{m}']} \mathbf{P}' \left[\left(\bigotimes_{m \in \mathbf{m}' \cap \mathbf{m}} (\mathbf{r}^{(m)} + \mathbf{a}^{(m)}) \right) \right. \right. \\
&\quad \left. \left. \otimes \left(\bigotimes_{m \in \mathbf{m}' \setminus \mathbf{m}} \mathbf{r}^{(m)} \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}'} \mathbf{r}^{(m)} \right) \right] \right) \\
&= \hat{\mathbf{L}}\mathbf{\Gamma}^\top \left(\sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \mathbf{P}' \left[\left(\bigotimes_{m \in \mathbf{m}' \cap \mathbf{m}} \mathbf{M}^{(m)} (\mathbf{r}^{(m)} + \mathbf{a}^{(m)}) \right) \right. \right. \\
&\quad \left. \left. \otimes \left(\bigotimes_{m \in \mathbf{m}' \setminus \mathbf{m}} \mathbf{M}^{(m)} \mathbf{r}^{(m)} \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}'} \mathbf{r}^{(m)} \right) \right] \right) \\
&= \hat{\mathbf{L}}\mathbf{\Gamma}^\top \left(\sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \mathbf{P}' \left[\left(\bigotimes_{m \in \mathbf{m}' \cap \mathbf{m}} (\mathbf{M}_{:0}^{(m)} + \mathbf{M}_{:a}^{(m)}) \right) \right. \right. \\
&\quad \left. \left. \otimes \left(\bigotimes_{m \in \mathbf{m}' \setminus \mathbf{m}} \mathbf{M}_{:0}^{(m)} \right) \otimes \left(\bigotimes_{m \notin \mathbf{m}'} \mathbf{r}^{(m)} \right) \right] \right). \tag{H7}
\end{aligned}$$

The order of the Kronecker factors may change at the third equality, corresponding to a new permutation $\mathbf{P}'$. The notation $\mathbf{M}_{:j}^{(m)}$ means the j th column of $\mathbf{M}^{(m)}$. In practice, the computation proceeds as follows: For each kernel term $\mathbf{m}' \in \mathcal{M}_{\text{kernel}}$, a Kronecker product of matrix columns is formed according to (H7). The Kronecker product is then partitioned according to MCs and added to an intermediary vector. If $\mathbf{m}' \notin \mathcal{M}_{\text{grid}}$ then the Kronecker product may generate out-of-space MCs, which should be skipped during computation due to the presence of the chopping matrix $\mathbf{\Gamma}^\top$. Finally, $\hat{\mathbf{L}}$ is applied to the intermediary vector to obtain the result.

To derive a definite complexity, we consider identical kernel and grid MCRs ($\mathcal{M}_{\text{kernel}} = \mathcal{M}_{\text{grid}}$) defined by a maximum interaction order or cut level, α . We also assume, for simplicity, that $n_1 = \dots = n_d = n$. Each of the dominating terms in (H7) consists of a Kronecker product of α vectors of size n , which has a cost of no more than αn^α . The number of α -terms is

$$\binom{D}{\alpha} \sim \mathcal{O}\left(\frac{D^\alpha}{\alpha!}\right) \tag{H8}$$

so the overall complexity of computing the sum is

$$\mathcal{O}\left(\alpha \frac{D^\alpha n^\alpha}{\alpha!}\right) = \mathcal{O}(\alpha \hat{N}), \tag{H9}$$

which is also the complexity of the subsequent MVP by $\hat{\mathbf{L}}$ (see Appendix G.2). We conclude that a single column of $\hat{\mathbf{K}}$ can be computed at a cost of $\mathcal{O}(\alpha \hat{N})$.

I Computing the diagonal of $\hat{\mathbf{K}}$

The pivoted Cholesky algorithm (see Appendix A.1) requires the main diagonal of the kernel matrix. We start by recalling the complete-grid kernel as written in (14):

$$\mathbf{K} = \sum_{\mathbf{m} \in \mathcal{M}_{\text{kernel}}} \sigma_{|\mathbf{m}|}^2 \mathbf{K}^{[\mathbf{m}]} \mathbf{J}^{[-\mathbf{m}]}. \quad (\text{I1})$$

The diagonal of $\hat{\mathbf{K}}$ consists of the elements $\hat{K}_{\mathbf{i}\mathbf{i}} = K_{\mathbf{i}\mathbf{i}}$ for $\mathbf{i} \in \hat{\mathcal{I}}$. It is convenient to rewrite each multi-index as $\mathbf{i} \cong (\mathbf{m}, \mathbf{a})$ (see Definition C.2), in which case

$$\begin{aligned} K_{\mathbf{i}\mathbf{i}} &= \left[\sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \mathbf{K}^{[\mathbf{m}']} \mathbf{J}^{[-\mathbf{m}']} \right]_{\mathbf{i}\mathbf{i}} \\ &= \sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \left(\prod_{m \in \mathbf{m}' \cap \mathbf{m}} K_{a^m a^m}^{(m)} \right) \left(\prod_{m \in \mathbf{m}' \setminus \mathbf{m}} K_{00}^{(m)} \right) \\ &= \sum_{\mathbf{m}'} \sigma_{|\mathbf{m}'|}^2 \left(\prod_{m \in \mathbf{m}' \cap \mathbf{m}} \frac{K_{a^m a^m}^{(m)}}{K_{00}^{(m)}} \right) \left(\prod_{m \in \mathbf{m}'} K_{00}^{(m)} \right). \end{aligned} \quad (\text{I2})$$

We have assumed $K_{00}^{(m)} \neq 0$. The intersection $\mathbf{s} = \mathbf{m}' \cap \mathbf{m}$ of $\mathbf{m}' \in \mathcal{M}_{\text{kernel}}$ and $\mathbf{m} \in \mathcal{M}_{\text{grid}}$ plays a central role in the following. The crucial idea is to group the terms in (I2) according to $\mathbf{s}$. We start by noting that the possible intersections are exactly the subsets of $\mathbf{m}$, i.e. $\mathbf{s} \subseteq \mathbf{m}$. Next, we define the set of kernel MCs with a prescribed intersection as

$$\mathcal{S}(\mathbf{m}, \mathbf{s}) = \{\mathbf{m}' \in \mathcal{M}_{\text{kernel}} \mid \mathbf{m}' \cap \mathbf{m} = \mathbf{s}\}. \quad (\text{I3})$$

With this notation,

$$\begin{aligned} K_{\mathbf{i}\mathbf{i}} &= \sum_{\mathbf{s} \subseteq \mathbf{m}} \sum_{\mathbf{m}' \in \mathcal{S}(\mathbf{m}, \mathbf{s})} \sigma_{|\mathbf{m}'|}^2 \left(\prod_{m \in \mathbf{s}} \frac{K_{a^m a^m}^{(m)}}{K_{00}^{(m)}} \right) \left(\prod_{m \in \mathbf{m}'} K_{00}^{(m)} \right) \\ &= \sum_{\mathbf{s} \subseteq \mathbf{m}} \left(\prod_{m \in \mathbf{s}} \frac{K_{a^m a^m}^{(m)}}{K_{00}^{(m)}} \right) \sum_{\mathbf{m}' \in \mathcal{S}(\mathbf{m}, \mathbf{s})} \sigma_{|\mathbf{m}'|}^2 \left(\prod_{m \in \mathbf{m}'} K_{00}^{(m)} \right). \end{aligned} \quad (\text{I4})$$

The outer sum is manageable given that $|\mathbf{m}|$ is small, while the inner sum involves only products of reference base kernel elements, $K_{00}^{(m)}$. As it stands, however, the inner sum must be computed for every combination of $\mathbf{m}$ and $\mathbf{s}$, which would be costly.

To get started, we define some notation for the inner sum:

$$X^{(\mathbf{m}, \mathbf{s})} = \sum_{\mathbf{m}' \in \mathcal{S}(\mathbf{m}, \mathbf{s})} P^{(\mathbf{m}')}, \quad (\text{I5})$$

$$P^{(\mathbf{m}')} = \sigma_{|\mathbf{m}'|}^2 \left(\prod_{m \in \mathbf{m}'} K_{00}^{(m)} \right). \quad (\text{I6})$$

The computational problem is the simultaneous dependency on $\mathbf{m}$ and $\mathbf{s}$. In order to eventually remove this problem, we let

$$\mathcal{S}(\mathbf{s}) = \{\mathbf{m}' \in \mathcal{M}_{\text{kernel}} \mid \mathbf{s} \subseteq \mathbf{m}'\}, \quad (\text{I7})$$

which is the set of all kernel MCs containing $\mathbf{s}$. The idea is now to group the elements of $\mathcal{S}(\mathbf{s})$ in a convenient way. To that end, choose a fixed $\mathbf{m} \in \mathcal{M}_{\text{grid}}$ such that $\mathbf{s} \subseteq \mathbf{m}$ and observe that

$$\mathbf{s} \subseteq \mathbf{m}, \mathbf{s} \subseteq \mathbf{m}' \Rightarrow \mathbf{s} \subseteq \mathbf{m}' \cap \mathbf{m}. \quad (\text{I8})$$

On the other hand, $\mathbf{m}' \cap \mathbf{m} \subseteq \mathbf{m}$ so

$$\mathbf{s} \subseteq (\mathbf{m}' \cap \mathbf{m}) \subseteq \mathbf{m}. \quad (\text{I9})$$

We conclude that the intersection $\mathbf{s}' = \mathbf{m}' \cap \mathbf{m}$ is at least $\mathbf{s}$ and at most $\mathbf{m}$. This implies the following partitioning of $\mathcal{S}(\mathbf{s})$ for $\mathbf{s} \subseteq \mathbf{m}$:

$$\mathcal{S}(\mathbf{s}) = \{\mathbf{m}' \in \mathcal{M}_{\text{kernel}} \mid \mathbf{s} \subseteq \mathbf{m}'\} = \bigcup_{\mathbf{s} \subseteq \mathbf{s}' \subseteq \mathbf{m}} \{\mathbf{m}' \in \mathcal{M}_{\text{kernel}} \mid \mathbf{m}' \cap \mathbf{m} = \mathbf{s}'\} = \bigcup_{\mathbf{s} \subseteq \mathbf{s}' \subseteq \mathbf{m}} \mathcal{S}(\mathbf{m}, \mathbf{s}'). \quad (\text{I10})$$

In turn, we find that

$$Y^{(\mathbf{s})} \equiv \sum_{\mathbf{m}' \in \mathcal{S}(\mathbf{s})} P^{(\mathbf{m}')} = \sum_{\mathbf{s} \subseteq \mathbf{s}' \subseteq \mathbf{m}} \left(\sum_{\mathbf{m}' \in \mathcal{S}(\mathbf{m}, \mathbf{s}')} P^{(\mathbf{m}')} \right) = \sum_{\mathbf{s} \subseteq \mathbf{s}' \subseteq \mathbf{m}} X^{(\mathbf{m}, \mathbf{s}')}, \quad (\text{I11})$$

which is in fact a triangular system of equations. Importantly, $Y^{(\mathbf{s})}$ does not depend on $\mathbf{m}$ and can thus be precomputed once before computing the entire diagonal.

To see the triangular structure, we examine a concrete example: Let $\mathbf{m} = (1, 2, 3)$ and consider (I11) for all $\mathbf{s} \subseteq \mathbf{m}$. Omitting commas between integers and writing $X^{(\mathbf{s}')}$ for $X^{(\mathbf{m}, \mathbf{s}')}$, we find that

$$\begin{bmatrix} Y^{(123)} \\ Y^{(23)} \\ Y^{(13)} \\ Y^{(12)} \\ Y^{(3)} \\ Y^{(2)} \\ Y^{(1)} \\ Y^{()} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} X^{(123)} \\ X^{(23)} \\ X^{(13)} \\ X^{(12)} \\ X^{(3)} \\ X^{(2)} \\ X^{(1)} \\ X^{()} \end{bmatrix}. \quad (\text{I12})$$

The dimension of the system is $2^{|\mathbf{m}|}$, so it can be solved using ordinary forward substitution with complexity $\mathcal{O}(4^{|\mathbf{m}|})$. It is, however, possible to derive an $\mathcal{O}(3^{|\mathbf{m}|})$ algorithm by taking advantage of the special structure. We start by slightly rearranging (I11):

$$X^{(\mathbf{m}, \mathbf{s})} = Y^{(\mathbf{s})} - \sum_{\mathbf{s} \subset \mathbf{s}' \subseteq \mathbf{m}} X^{(\mathbf{m}, \mathbf{s}')}. \quad (\text{I13})$$

This suggests the following algorithm:

Algorithm 5 Given $\mathbf{m} \in \mathcal{M}_{\text{grid}}$, compute $X^{(\mathbf{m}, \mathbf{s})}$ for all $\mathbf{s} \subseteq \mathbf{m}$.

```

for  $\mathbf{s} \subseteq \mathbf{m}$  do
   $X^{(\mathbf{m}, \mathbf{s})} = Y^{(\mathbf{s})}$ 
for  $\mathbf{s} \subseteq \mathbf{m}$  do ▷ Loop in descending order
  for  $\mathbf{s}' \subset \mathbf{s}$  do
     $X^{(\mathbf{m}, \mathbf{s}')} -= X^{(\mathbf{s})}$ 

```

Using the binomial theorem, the number of loop iterations in the nested (dominating) loop is

$$\sum_{k=0}^{|\mathbf{m}|} \binom{|\mathbf{m}|}{k} \sum_{l=0}^{k-1} \binom{k}{l} = \sum_{k=0}^{|\mathbf{m}|} \binom{|\mathbf{m}|}{k} \left(-1 + \sum_{l=0}^k \binom{k}{l} \right) = 3^{|\mathbf{m}|} - 2^{|\mathbf{m}|}, \quad (\text{I14})$$

leading to a complexity of $\mathcal{O}(3^{|\mathbf{m}|})$.

The following algorithm describes how to precompute $Y^{(\mathbf{s})}$ for all $\mathbf{s} \in \mathcal{M}_{\text{grid}}$:

Algorithm 6 Compute $Y^{(s)}$ for all $s \in \mathcal{M}_{\text{grid}}$. Assumes $\mathcal{M}_{\text{kernel}} \subseteq \mathcal{M}_{\text{grid}}$.

```

for  $s \in \mathcal{M}_{\text{grid}}$  do
   $Y^{(s)} = 0$ 
  for  $s' \in \mathcal{M}_{\text{kernel}}$  do
     $P = \sigma_{|s|}^2$ 
    for  $m \in s$  do
       $P *= K_{00}^{(m)}$ 
    for  $s' \subseteq s$  do
       $Y^{(s')} += P$ 

```

Assuming simple kernel and grid MCRs ($\mathcal{M}_{\text{kernel}} = \mathcal{M}_{\text{grid}}$) with a maximum interaction order or cut level, α , we see that the nested loop over s and s' dominates with a complexity of

$$\mathcal{O}\left(\frac{D^\alpha 2^\alpha}{\alpha!}\right). \quad (\text{I15})$$

We now have the tools to compute the entire diagonal:

Algorithm 7 Compute K_{ii} for all $i \in \hat{\mathcal{I}}$. Equivalently, compute $K_{(\mathbf{m}, \mathbf{a})(\mathbf{m}, \mathbf{a})}$ for all $\mathbf{m} \in \mathcal{M}_{\text{grid}}$ and $\mathbf{a} \in \mathcal{A}^{(\mathbf{m})}$ (see Definition C.1). Assumes $\mathcal{M}_{\text{kernel}} \subseteq \mathcal{M}_{\text{grid}}$.

```

Compute  $Y^{(s)}$  for all  $s \in \mathcal{M}_{\text{grid}}$  using Algorithm 6.
for  $\mathbf{m} \in \mathcal{M}_{\text{grid}}$  do
  Compute  $X^{(\mathbf{m}, s)}$  for all  $s \subseteq \mathbf{m}$  using Algorithm 5.
  for  $s \subseteq \mathbf{m}$  do
    for  $\mathbf{a} \in \mathcal{A}^{(\mathbf{m})}$  do
       $K_{(\mathbf{m}, \mathbf{a})(\mathbf{m}, \mathbf{a})} += \left( \prod_{m \in s} \frac{K_{a^m a^m}^m}{K_{00}^{(m)}} \right) X^{(\mathbf{m}, s)}$ 

```

Taking each loop into account (and assuming $n_1 = \dots = n_d = n$ for simplicity), we find a complexity of

$$\mathcal{O}\left(\alpha 2^\alpha \frac{D^\alpha n^\alpha}{\alpha!}\right) = \mathcal{O}(\alpha 2^\alpha \hat{N}). \quad (\text{I16})$$

This should be compared to the MVP by $\hat{\mathbf{K}}$, which has a cost of $\mathcal{O}(n\alpha\hat{N})$. The α -scaling related to the diagonal is clearly more severe, but at the same time we expect α to be relatively small. In practice, we thus anticipate that 2^α is comparable to n , leading to roughly similar costs for the kernel diagonal and the kernel MVP. It should also be recalled that the diagonal is computed only once in each optimization cycle (when the preconditioner is constructed), whereas the kernel MVP is computed a large number times.

J The preconditioner trace term

In order to reduce the variance of the trace term estimate [12] we must be able to compute the preconditioner trace term, $\text{tr}(\mathbf{P}^{-1}\partial\mathbf{P}/\partial\theta)$, given that $\mathbf{P}$ is a pivoted Cholesky preconditioner of rank k :

$$\mathbf{P} = \mathbf{Z}\mathbf{Z}^\top + \sigma^2\mathbf{I}, \quad \mathbf{Z} \in \mathbb{R}^{\hat{N} \times k}. \quad (\text{J1})$$

As a working example we will consider the length scale trace terms, i.e. $\text{tr}(\mathbf{P}^{-1}\partial\mathbf{P}/\partial\ell^{(m)})$ for $m = 1, \dots, D$. In practice, we may not use an independent length scale for each dimension, so this is a worst-case example that we should nevertheless be able to handle.

The simplest option requires an explicit representation of the derivative of the pivoted Cholesky decomposition. Considering any (non-noise) hyperparameter θ , we have

$$\frac{\partial\mathbf{P}}{\partial\theta} = \frac{\partial\mathbf{Z}}{\partial\theta}\mathbf{Z}^\top + \mathbf{Z}\frac{\partial\mathbf{Z}^\top}{\partial\theta} = \mathbf{X}\mathbf{Z}^\top + \mathbf{Z}\mathbf{X}^\top, \quad \mathbf{X} \in \mathbb{R}^{\hat{N} \times k}. \quad (\text{J2})$$

The derivative Cholesky vectors $\mathbf{X}$ can be obtained by manual [40, 41] or automatic [42] differentiation of the pivoted Cholesky algorithm, but storage alone costs $\mathcal{O}(kp\hat{N})$ where p is the number of hyperparameters. For the length scales in our working example, this translates to $\mathcal{O}(kD\hat{N})$ or $\mathcal{O}(kD^{\alpha+1}n^\alpha/\alpha!)$ if the kernel and grid MCRs are simple. To avoid the D -scaling of $\mathcal{O}(D^{\alpha+1})$, which does not occur elsewhere in our code, we will use the connection between the pivoted Cholesky factorization and the column Nyström approximation [see, e.g., 31, Property 2.1]. Using MATLAB notation we have

$$\mathbf{Z}\mathbf{Z}^\top = \hat{\mathbf{K}}(:, \mathcal{P})\hat{\mathbf{K}}(\mathcal{P}, \mathcal{P})^{-1}\hat{\mathbf{K}}(:, \mathcal{P})^\top. \quad (\text{J3})$$

where $\mathcal{P}$ denotes the set of pivot indices as determined by the pivoted Cholesky algorithm. The pivot columns and the pivot submatrix can be written as

$$\mathbf{A} = \hat{\mathbf{K}}(:, \mathcal{P}) = \hat{\mathbf{K}}\boldsymbol{\Delta} \in \mathbb{R}^{\hat{N} \times k}, \quad (\text{J4})$$

$$\mathbf{B} = \hat{\mathbf{K}}(\mathcal{P}, \mathcal{P}) = \boldsymbol{\Delta}^\top \hat{\mathbf{K}}\boldsymbol{\Delta} \in \mathbb{R}^{k \times k}, \quad (\text{J5})$$

where $\boldsymbol{\Delta} \in \mathbb{R}^{\hat{N} \times k}$ is a chopping matrix that selects the pivot indices, i.e. the columns of $\boldsymbol{\Delta}$ are simply standard basis vectors $\hat{\mathbf{e}}_{\mathbf{i}}$ with $\mathbf{i} \in \mathcal{P} \subseteq \hat{\mathcal{I}}$. The expressions involving chopping will be useful later on, since they allow the derivatives of $\mathbf{A}$ and $\mathbf{B}$ to be computed via fast MVPs. The pivot columns, $\mathbf{A}$, are already computed by the pivoted Cholesky algorithm, and the pivot submatrix $\mathbf{B}$ is easily obtained from $\mathbf{A}$ by selecting the appropriate rows. Taking the derivative of (J3) with respect to a non-noise hyperparameter θ , we obtain

$$\begin{aligned} \frac{\partial\mathbf{P}}{\partial\theta} &= \frac{\partial\mathbf{Z}\mathbf{Z}^\top}{\partial\theta} = \frac{\partial\mathbf{A}\mathbf{B}^{-1}\mathbf{A}^\top}{\partial\theta} = \frac{\partial\mathbf{A}}{\partial\theta}\mathbf{B}^{-1}\mathbf{A}^\top + \mathbf{A}\frac{\partial\mathbf{B}^{-1}}{\partial\theta}\mathbf{A}^\top + \mathbf{A}\mathbf{B}^{-1}\frac{\partial\mathbf{A}^\top}{\partial\theta} \\ &= \frac{\partial\mathbf{A}}{\partial\theta}\mathbf{B}^{-1}\mathbf{A}^\top - \mathbf{A}\mathbf{B}^{-1}\frac{\partial\mathbf{B}}{\partial\theta}\mathbf{B}^{-1}\mathbf{A}^\top + \mathbf{A}\mathbf{B}^{-1}\frac{\partial\mathbf{A}^\top}{\partial\theta}. \end{aligned} \quad (\text{J6})$$

The Nyström format (J3) can also be used to express $\mathbf{P}^{-1}$ via the matrix inverse lemma, which would remove the need to store the Cholesky vectors, $\mathbf{Z}$. We have found, however, that the Cholesky format produces more accurate solves, and we will thus use Cholesky for $\mathbf{P}^{-1}$ and Nyström for $\partial\mathbf{P}/\partial\theta$ when computing the trace terms. Before proceeding we recall (A5):

$$\mathbf{P}^{-1} = \frac{1}{\sigma^2} (\mathbf{I}_{\hat{N}} - \mathbf{Z}\mathbf{Q}^{-1}\mathbf{Z}^\top). \quad (\text{J7})$$

It is now a simple matter to combine (J6) and (J7). Using the invariance of the trace under transposition and cyclic shifts, one finds the expression

$$\sigma^2 \text{tr} \left(\mathbf{P}^{-1} \frac{\partial\mathbf{P}}{\partial\theta} \right) = 2 \text{tr} \left(\mathbf{X}^\top \frac{\partial\mathbf{A}}{\partial\theta} \right) + \text{tr} \left(\mathbf{Y}^\top \frac{\partial\mathbf{B}}{\partial\theta} \right) \quad (\text{J8})$$

with the definitions

$$\mathbf{X}^\top = (\mathbf{B}^{-1}\mathbf{A}^\top) - ((\mathbf{B}^{-1}\mathbf{A}^\top)\mathbf{Z})(\mathbf{Q}^{-1}\mathbf{Z}^\top) \in \mathbb{R}^{k \times \hat{N}}, \quad (\text{J9a})$$

$$\mathbf{Y}^\top = ((\mathbf{B}^{-1}\mathbf{A}^\top)\mathbf{Z})((\mathbf{Q}^{-1}\mathbf{Z}^\top)(\mathbf{B}^{-1}\mathbf{A}^\top)^\top) - (\mathbf{B}^{-1}\mathbf{A}^\top)(\mathbf{B}^{-1}\mathbf{A}^\top)^\top \in \mathbb{R}^{k \times k}. \quad (\text{J9b})$$

Parentheses indicate the order of evaluation used in our code (expressions that occur multiple times should of course be computed once and then reused). Computing $\mathbf{X}$ and $\mathbf{Y}$ from $\mathbf{B}$, $\mathbf{Q}$, $\mathbf{A}$ and $\mathbf{Z}$ has a complexity of $\mathcal{O}(k^2\hat{N})$.

We still need to handle the derivatives. Using $\mathbf{A} = \hat{\mathbf{K}}\mathbf{\Delta} = \hat{\mathbf{L}}\hat{\mathbf{M}}\hat{\mathbf{U}}\mathbf{\Delta}$ and $\mathbf{B} = \mathbf{\Delta}^\top\hat{\mathbf{K}}\mathbf{\Delta} = \mathbf{\Delta}^\top\hat{\mathbf{L}}\hat{\mathbf{M}}\hat{\mathbf{U}}\mathbf{\Delta}$, we can write the matrices on the right-hand side of (J8) as

$$\mathbf{X}^\top \frac{\partial \mathbf{A}}{\partial \theta} = \mathbf{X}^\top \hat{\mathbf{L}} \frac{\partial \hat{\mathbf{M}}}{\partial \theta} \hat{\mathbf{U}} \mathbf{\Delta} = (\hat{\mathbf{U}} \mathbf{X})^\top \left(\frac{\partial \hat{\mathbf{M}}}{\partial \theta} \hat{\mathbf{U}} \mathbf{\Delta} \right), \quad (\text{J10a})$$

$$\mathbf{Y}^\top \frac{\partial \mathbf{B}}{\partial \theta} = \mathbf{Y}^\top \mathbf{\Delta}^\top \hat{\mathbf{L}} \frac{\partial \hat{\mathbf{M}}}{\partial \theta} \hat{\mathbf{U}} \mathbf{\Delta} = (\hat{\mathbf{U}}(\mathbf{\Delta} \mathbf{Y}))^\top \left(\frac{\partial \hat{\mathbf{M}}}{\partial \theta} \hat{\mathbf{U}} \mathbf{\Delta} \right). \quad (\text{J10b})$$

The cost of $\mathbf{\Delta} \mathbf{Y}$ is only $\mathcal{O}(k\hat{N})$ due to the sparsity of $\mathbf{\Delta}$. The matrices $\hat{\mathbf{U}} \mathbf{X}$ and $\hat{\mathbf{U}}(\mathbf{\Delta} \mathbf{Y})$ can each be obtained at a cost of $\mathcal{O}(k\alpha\hat{N})$ by computing fast MVPs with $\hat{\mathbf{U}}$ (see Appendix G.2). The columns of $\mathbf{\Delta}$ are simply standard basis vectors, so we can use the techniques of Appendices H and G.4 to compute the sparse columns of $(\partial \hat{\mathbf{M}} / \partial \theta) \hat{\mathbf{U}} \mathbf{\Delta} \in \mathbb{R}^{\hat{N} \times k}$ directly. The diagonals of $\mathbf{X}^\top (\partial \mathbf{A} / \partial \theta)$ and $\mathbf{Y}^\top (\partial \mathbf{B} / \partial \theta)$, which are sufficient for computing the traces, require only k sparse dot products each.

We have now established a precomputation phase where $\mathbf{X}$, $\mathbf{Y}$, $\hat{\mathbf{U}} \mathbf{X}$ and $\hat{\mathbf{U}}(\mathbf{\Delta} \mathbf{Y})$ are computed with leading-order complexity $\mathcal{O}(k^2\hat{N})$ (assuming $k > \alpha$, which is almost always the case in practice). This phase is followed by computing, for each θ , the k columns of $(\partial \hat{\mathbf{M}} / \partial \theta) \hat{\mathbf{U}} \mathbf{\Delta}$ as well as k sparse dot products.

Returning now to our working example, we use the fact that each column of $(\partial \hat{\mathbf{M}} / \partial \ell^{(m)}) \hat{\mathbf{U}} \mathbf{\Delta}$ has a cost of $\mathcal{O}(\alpha^2 \hat{N} / D)$, while each sparse dot product scales as $\mathcal{O}(\alpha \hat{N} / D)$ (see Appendix G.4). Computing the traces for all $m = 1, \dots, D$ thus has a total cost of $\mathcal{O}(k\alpha^2 \hat{N})$.

K Overall structure of the code

Having covered all components necessary to do MVP-based GPR, a summary is in place. The computation proceeds in two main phases: Hyperparameter optimization (N_{opt} cycles) and prediction ($\hat{N}_*$ points). At the beginning of each optimization cycle, the preconditioner is constructed and a number of preconditioner-related quantities are precomputed. This is followed by estimation of the MLL,

$$\mathcal{L} = -\frac{1}{2} \left(\hat{\mathbf{y}}^\top \hat{\boldsymbol{\alpha}} + \log |\hat{\mathbf{C}}| + \hat{N} \log 2\pi \right), \quad (\text{K1})$$

and the MLL gradient,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{1}{2} \hat{\boldsymbol{\alpha}}^\top \frac{\partial \hat{\mathbf{C}}}{\partial \theta} \hat{\boldsymbol{\alpha}} - \frac{1}{2} \text{tr} \left(\hat{\mathbf{C}}^{-1} \frac{\partial \hat{\mathbf{C}}}{\partial \theta} \right). \quad (\text{K2})$$

The dominating part of Eqs. (K1) and (K2) is the modified CG algorithm needed for estimating the log-determinant and the trace terms. Each CG iteration spends a kernel MVP, an inverse preconditioner MVP and a few dot products (which we will ignore when analyzing the cost).

The predictive mean and variance for a single test point is

$$\mu = \hat{\mathbf{k}}_*^\top \hat{\boldsymbol{\alpha}} \quad (\text{K3})$$

$$\Sigma = K_{**} - \hat{\mathbf{k}}_*^\top \hat{\mathbf{C}}^{-1} \hat{\mathbf{k}}_* \quad (\text{K4})$$

where $\hat{\mathbf{k}}_* \in \mathbb{R}^{\hat{N}}$ is a column of the training–test cross-covariance matrix and the scalar K_{**} is the test–test covariance. Again, the dominating part is the CG algorithm required for the matrix inverse. The training–test column can be computed by a slightly modified version of the algorithm in Appendix H at a cost of $\mathcal{O}(\alpha \hat{N})$.

We use the symbols N_{opt} , N_{probe} and N_{CG} for the number of optimization cycles, probe vectors and CG iterations. The preconditioner rank is k , and it is assumed that $k < \hat{N}$, $\alpha < k$ and $N_{\text{CG}} < \hat{N}$. In addition, we only consider the MLL gradient with respect to the length scales (the order variance gradients are not dominating). With these assumptions in place, we are ready to summarize the various steps in more detail. The summary is not fully exhaustive – it is only intended to cover the parts that are conceptually or computationally significant.

- Phase 1: Hyperparameter optimization (N_{opt} cycles)
 - Preconditioner
 - * Construct $\mathbf{P}$ and precompute preconditioner-related quantities: $\mathcal{O}(k^2 \hat{N})$.
 - * Compute the log-determinant of $\mathbf{P}$: $\mathcal{O}(k)$.
 - * Prepare the preconditioner trace terms: $\mathcal{O}(k^2 \hat{N})$
 - * Compute the preconditioner trace terms: $\mathcal{O}(k \alpha^2 \hat{N})$
 - Estimates of MLL and MLL gradient
 - * Quadratic terms: $\mathcal{O}(n \alpha^2 \hat{N})$.
 - * Stochastic trace estimation: $\mathcal{O}(N_{\text{probe}} N_{\text{CG}} (n \alpha \hat{N} + k \hat{N}))$
- Phase 2: Prediction ($\hat{N}_*$ points)
 - Compute training–test column: $\mathcal{O}(\alpha \hat{N})$.
 - Compute predictive mean: $\mathcal{O}(\hat{N})$.
 - Compute predictive variance: $\mathcal{O}(N_{\text{CG}} (n \alpha \hat{N} + k \hat{N}))$.

Looking at the summary, we see that the most expensive parts are the stochastic trace estimation and the predictive variances with complexities

$$\mathcal{O}(N_{\text{opt}} N_{\text{probe}} N_{\text{CG}} (n \alpha \hat{N} + k \hat{N})) \quad \text{and} \quad \mathcal{O}(\hat{N}_* N_{\text{CG}} (n \alpha \hat{N} + k \hat{N})),$$

respectively. Ideally, the parameters N_{opt} , N_{probe} , N_{CG} and k do not depend on $\hat{N}$, in which case the complexities just mentioned essentially reduce to the complexity of the kernel MVP, i.e. $\mathcal{O}(n \alpha \hat{N})$.

Whether this happens in practice depends chiefly on the conditioning of the kernel, which does not depend on $\hat{N}$ as such but rather on the nature of the dataset and the value of the hyperparameters. Noise plays a particularly important role in this regard, since larger noise improves the conditioning of the kernel.

Irrespective of the exact scaling, the MVP-based framework has the great advantage that it is easily parallelized: Trace estimation can be parallelized over probe vectors and predictions over test points.

L Empirical complexity of the kernel matrix-vector product

All benchmarks were run using standard double precision on a single core of an Intel Xeon Platinum 8358 processor. The benchmarks were run using Google Benchmark,² which dynamically determines an appropriate number of iterations to ensure stable timings. The minimum warm-up time and minimum running time were set to 2 s.

We mention in Section 6.1 that the empirical D -complexities are slightly too large, in particular for $\alpha = 4$, which we take as a sign that the asymptotic region is not yet fully reached. This hypothesis has been investigated for each combination of α and n by including only the lower or upper half of the data during fitting. In all cases, the latter fit has a smaller slope (closer to the theoretical slope). The deviations are anyway rather small, so we take Figure 2a as confirmation that the D -complexity is correct.

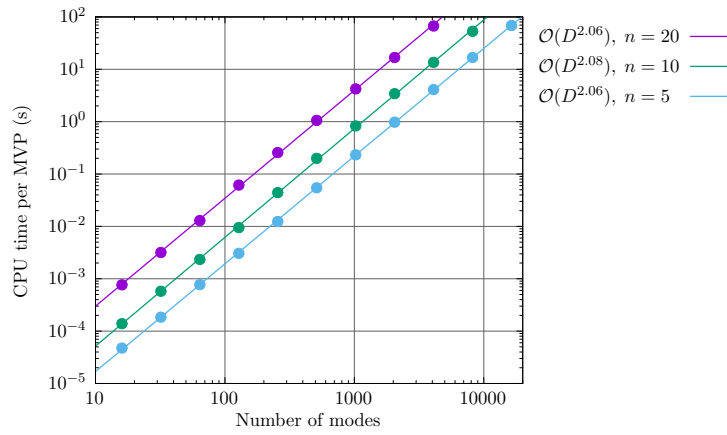


Figure L1: CPU times for the kernel MVP with $\alpha = 2$ and $n = 5, 10, 20$.

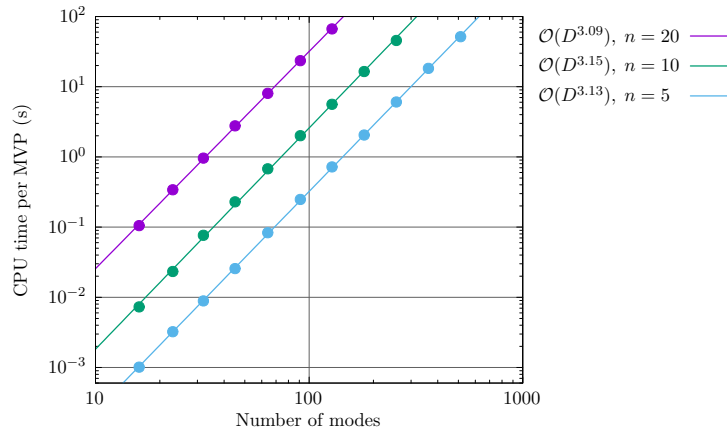


Figure L2: CPU times for the kernel MVP with $\alpha = 3$ and $n = 5, 10, 20$.

²Version 1.9.4, Apache License 2.0, available at <https://github.com/google/benchmark>.

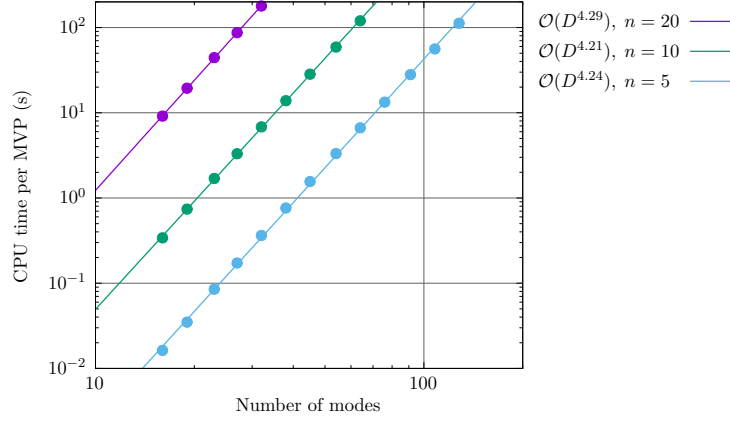


Figure L3: CPU times for the kernel MVP with $\alpha = 4$ and $n = 5, 10, 20$.

Table L1: Benchmark data for $\alpha = 2$ and $n = 5, 10, 20$. Column five indicates the memory consumption corresponding to a single vector of size $\hat{N}$ (assuming standard double precision).

α	n	D	$\hat{N}$	Memory	CPU time (s)
2	5	16	1.99×10^3	15.9 kB	4.74×10^{-5}
		32	8.07×10^3	64.5 kB	1.85×10^{-4}
		64	3.25×10^4	260.1 kB	7.70×10^{-4}
		128	1.31×10^5	1.0 MB	3.06×10^{-3}
		256	5.23×10^5	4.2 MB	1.24×10^{-2}
		512	2.10×10^6	16.8 MB	5.45×10^{-2}
		1024	8.38×10^6	67.1 MB	2.33×10^{-1}
		2048	3.35×10^7	268.4 MB	9.78×10^{-1}
		4096	1.34×10^8	1.1 GB	4.11
		8192	5.37×10^8	4.3 GB	1.68×10^1
2	10	16	9.87×10^3	78.9 kB	1.39×10^{-4}
		32	4.05×10^4	323.7 kB	5.77×10^{-4}
		64	1.64×10^5	1.3 MB	2.34×10^{-3}
		128	6.60×10^5	5.3 MB	9.49×10^{-3}
		256	2.65×10^6	21.2 MB	4.42×10^{-2}
		512	1.06×10^7	84.8 MB	1.99×10^{-1}
		1024	4.24×10^7	339.5 MB	8.28×10^{-1}
		2048	1.70×10^8	1.4 GB	3.42
		4096	6.79×10^8	5.4 GB	1.36×10^1
		8192	2.72×10^9	21.7 GB	5.31×10^1
2	20	16	4.36×10^4	349.0 kB	7.67×10^{-4}
		32	1.80×10^5	1.4 MB	3.18×10^{-3}
		64	7.29×10^5	5.8 MB	1.29×10^{-2}
		128	2.94×10^6	23.5 MB	6.16×10^{-2}
		256	1.18×10^7	94.3 MB	2.57×10^{-1}
		512	4.72×10^7	377.9 MB	1.06
		1024	1.89×10^8	1.5 GB	4.23
		2048	7.57×10^8	6.1 GB	1.68×10^1
		4096	3.03×10^9	24.2 GB	6.71×10^1

Table L2: Benchmark data for $\alpha = 3$ and $n = 5, 10, 20$. Column five indicates the memory consumption corresponding to a single vector of size $\hat{N}$ (assuming standard double precision).

α	n	D	$\hat{N}$	Memory	CPU time (s)
3	5	16	3.78×10^4	302.6 kB	1.01×10^{-3}
		23	1.17×10^5	939.9 kB	3.24×10^{-3}
		32	3.26×10^5	2.6 MB	8.88×10^{-3}
		45	9.24×10^5	7.4 MB	2.56×10^{-2}
		64	2.70×10^6	21.6 MB	8.32×10^{-2}
		91	7.84×10^6	62.7 MB	2.48×10^{-1}
		128	2.20×10^7	175.8 MB	7.19×10^{-1}
		181	6.25×10^7	499.7 MB	2.05
		256	1.77×10^8	1.4 GB	6.06
		362	5.03×10^8	4.0 GB	1.83×10^1
		512	1.43×10^9	11.4 GB	5.15×10^1
3	10	16	4.18×10^5	3.3 MB	7.29×10^{-3}
		23	1.31×10^6	10.5 MB	2.33×10^{-2}
		32	3.66×10^6	29.3 MB	7.65×10^{-2}
		45	1.04×10^7	83.4 MB	2.29×10^{-1}
		64	3.05×10^7	244.3 MB	6.77×10^{-1}
		91	8.89×10^7	711.2 MB	2.00
		128	2.50×10^8	2.0 GB	5.62
		181	7.10×10^8	5.7 GB	1.65×10^1
		256	2.02×10^9	16.1 GB	4.55×10^1
3	20	16	3.88×10^6	31.1 MB	1.05×10^{-1}
		23	1.22×10^7	97.9 MB	3.40×10^{-1}
		32	3.42×10^7	273.6 MB	9.60×10^{-1}
		45	9.77×10^7	781.5 MB	2.77
		64	2.87×10^8	2.3 GB	8.04
		91	8.35×10^8	6.7 GB	2.35×10^1
		128	2.34×10^9	18.8 GB	6.66×10^1

Table L3: Benchmark data for $\alpha = 4$ and $n = 5, 10, 20$. Column five indicates the memory consumption corresponding to a single vector of size $\hat{N}$ (assuming standard double precision).

α	n	D	$\hat{N}$	Memory	CPU time (s)
4	5	16	5.04×10^5	4.0 MB	1.63×10^{-2}
		19	1.06×10^6	8.5 MB	3.49×10^{-2}
		23	2.38×10^6	19.1 MB	8.50×10^{-2}
		27	4.69×10^6	37.5 MB	1.73×10^{-1}
		32	9.53×10^6	76.3 MB	3.63×10^{-1}
		38	1.94×10^7	155.6 MB	7.63×10^{-1}
		45	3.91×10^7	312.5 MB	1.56
		54	8.26×10^7	660.6 MB	3.32
		64	1.65×10^8	1.3 GB	6.66
		76	3.33×10^8	2.7 GB	1.33×10^1
		91	6.92×10^8	5.5 GB	2.80×10^1
		108	1.39×10^9	11.1 GB	5.60×10^1
		128	2.75×10^9	22.0 GB	1.12×10^2
4	10	16	1.24×10^7	98.9 MB	3.40×10^{-1}
		19	2.62×10^7	209.2 MB	7.40×10^{-1}
		23	5.94×10^7	475.3 MB	1.70
		27	1.17×10^8	938.5 MB	3.30
		32	2.40×10^8	1.9 GB	6.83
		38	4.91×10^8	3.9 GB	1.38×10^1
		45	9.88×10^8	7.9 GB	2.84×10^1
		54	2.09×10^9	16.7 GB	5.90×10^1
		64	4.20×10^9	33.6 GB	1.20×10^2
4	20	16	2.41×10^8	1.9 GB	9.14
		19	5.12×10^8	4.1 GB	1.94×10^1
		23	1.17×10^9	9.3 GB	4.43×10^1
		27	2.31×10^9	18.5 GB	8.71×10^1
		32	4.72×10^9	37.8 GB	1.79×10^2

M Additional computational details and results

M.1 Generating the datasets

All PES values (electronic energies) have been computed with the semi-empirical GFN2-xTB method [43] as implemented the xTB program³ [44]. First, the molecular geometries were optimized using GFN2-xTB with the ‘extreme’ convergence threshold (the energy is converged to 5×10^{-8} and the gradient norm to $5 \times 10^{-5} E_h/a_0$). Then, the Hessian was computed at the same level of theory. The Hessian was subsequently diagonalized by the MidasCpp program⁴ [45] to obtain mass-scaled normal coordinates. Finally, MidasCpp was used to generate the set of grid points and gather the electronic energies via an interface to the xTB program.

Table M1: Ten organic molecules with ten atoms ($D = 24$).

Name	Chemical formula
Butadiene	C_4H_6
DMSO	$(CH_3)_2SO$
Ethylamine	$CH_3CH_2NH_2$
Ethylene glycol	$(CH_2OH)_2$
Nitroethane	$CH_3CH_2NO_2$
Propanal	CH_3CH_2CHO
Pyrazine	$C_4H_4N_2$
Pyrrole	C_4H_4NH
Thioacetone	$(CH_3)_2CS$
Vinylformamide	C_3H_5NO

Table M2: 1D grids used for sampling the PESs. The grid points are given as fractions of the 1D grid bounds, which are chosen as the classical turning points of the 1D harmonic oscillator state with $v = 10$. In mass-scaled coordinates, the turnings points are given by $x_{TP}(v) = \pm \sqrt{2h(v + 1/2)/\omega}$. With a cut level of $\alpha = 3$, the training and test sets consist of $\hat{N} = 447\,265$ and $\hat{N}_* = 1\,604\,576$ points, respectively.

1D grid points											
Fine	-1.0	-0.8	-0.6	-0.4	-0.2	0.0	+0.2	+0.4	+0.6	+0.8	+1.0
Coarse	-1.0		-0.6		-0.2	0.0	+0.2		+0.6		+1.0

M.2 Kernel centering

Generally speaking, the base kernel centering depends on the distribution of input locations in the training set.^{5,15} Using the empirical distribution, which is discrete, one finds that centering amounts to the following inexpensive modification of the training–training, training–test and test–test base matrices:

$$\mathbf{K}^{(m)} \leftarrow \mathbf{K}^{(m)} - \frac{\mathbf{K}^{(m)} \mathbf{w}^{(m)} (\mathbf{w}^{(m)})^\top \mathbf{K}^{(m)}}{(\mathbf{w}^{(m)})^\top \mathbf{K}^{(m)} \mathbf{w}^{(m)}}, \quad (\text{M1a})$$

$$\mathbf{K}_*^{(m)} \leftarrow \mathbf{K}_*^{(m)} - \frac{\mathbf{K}^{(m)} \mathbf{w}^{(m)} (\mathbf{w}^{(m)})^\top \mathbf{K}_*^{(m)}}{(\mathbf{w}^{(m)})^\top \mathbf{K}^{(m)} \mathbf{w}^{(m)}}, \quad (\text{M1b})$$

$$\mathbf{K}_{**}^{(m)} \leftarrow \mathbf{K}_{**}^{(m)} - \frac{(\mathbf{K}_*^{(m)})^\top \mathbf{w}^{(m)} (\mathbf{w}^{(m)})^\top \mathbf{K}_*^{(m)}}{(\mathbf{w}^{(m)})^\top \mathbf{K}^{(m)} \mathbf{w}^{(m)}}. \quad (\text{M1c})$$

³Version 6.3.3, license CC BY-SA 4.0, available at <https://github.com/grimme-lab/xtb>.

⁴Version 2025.10.0, license LGPL 2.1, available at <https://source.coderefinery.org/midascpp/midascpp>.

The vector $\mathbf{w}^{(m)}$ contains the frequencies with which the 1D grid points for mode m occur in the training set. The first element of $\mathbf{w}^{(m)}$ is largest, since the reference is the most frequent point for every mode.

M.3 Initial values

Table M3: Initial values for the hyperparameter optimization. For each mode, the initial length scale is computed as twice the average distance between neighbouring 1D grid points. This distance is computed after the input data is standardized, so identical values are obtained for all modes and all molecules.

Hyperparameter	Initial value
σ_0^2	1.00×10^{-4}
σ_1^2	1.25×10^{-1}
σ_2^2	2.50×10^{-1}
σ_3^2	5.00×10^{-1}
$\ell^{(m)}$	2.77

M.4 Priors over hyperparameters

Initial experiments showed that the order variances, σ_k^2 , tend to become very large if the optimization is run without constraints. This leads to deterioration of the condition number of the kernel, which in turn reduces the overall computational efficiency. At the same time, we observed no significant gain in accuracy when the order variances were allowed to grow freely. To avoid this situation, we place Gamma(1.0, 0.1) priors on the order variances, similar to Lu et al. [15].

To avoid overfitting, we place a log-normal prior on each length scale as described by Hvarfner et al. [23]. Since the average distance between two randomly sampled points in a D -dimensional hypercube scales as $\sqrt{D}$, they recommend a prior of the form

$$\ell^{(m)} \sim \text{Lognormal}(\mu_0 + \log(\sqrt{D}), \sigma_0) \quad (\text{M2})$$

whose mean and mode also scale as $\sqrt{D}$. In our setup, the points are confined to low-dimensional subgrids/cuts rather than being distributed over the entire hypercube. Assuming a cut level of α , the effective dimensionality turns out to be $2\alpha < D$ (because two points on two non-overlapping α -cuts differ along exactly 2α dimensions). We therefore use

$$\ell^{(m)} \sim \text{Lognormal}(\mu_0 + \log(\sqrt{2\alpha}), \sigma_0) \quad (\text{M3})$$

with $\mu_0 = \sqrt{2}$ and $\sigma_0 = \sqrt{3}$.

To maintain the regularizing effect as the number of training points increases, we normalize the MLL before adding the prior terms. The objective function we optimize is thus

$$\mathcal{L}' = \frac{\mathcal{L}}{\widehat{N}} + \text{prior terms} \quad (\text{M4})$$

M.5 SVGP settings

All calculations were run with the same priors and noise as CUTS-GPR, though inputs were standardized to the range $[0, 1]$. Hyperparameters were initialized with default values and optimized using Adam with a learning rate of 0.1 ($\omega = 2$) or 0.01 ($\omega = 3$). The optimization was run until the change in objective function was less than 0.1 over 10 iterations. All other settings were left as default.

M.6 Trace estimation

Table M4: Stochastic estimates of the trace terms, $\text{tr}(\hat{\mathbf{C}}^{-1}\partial\hat{\mathbf{C}}/\partial\theta)$, for iteration 50 of the butadiene calculation. The standard error of the mean (SEM) is shown as a measure of uncertainty. Note that the SEM is approximate since it assumes a CG error of zero.

Hyperparameter	Estimate	SEM (%)
σ_0^2	$+3.99 \times 10^3$	518.00
σ_1^2	$+3.45 \times 10^3$	37.60
σ_2^2	$+7.47 \times 10^4$	0.72
σ_3^2	$+5.03 \times 10^5$	0.05
$\ell^{(1)}$	-8.38×10^3	0.47
$\ell^{(2)}$	-7.46×10^3	0.54
$\ell^{(3)}$	-1.10×10^4	0.42
$\ell^{(4)}$	-5.86×10^3	0.51
$\ell^{(5)}$	-8.02×10^3	0.43
$\ell^{(6)}$	-9.61×10^3	0.42
$\ell^{(7)}$	-9.96×10^3	0.39
$\ell^{(8)}$	-7.55×10^3	0.38
$\ell^{(9)}$	-8.59×10^3	0.41
$\ell^{(10)}$	-1.05×10^4	0.43
$\ell^{(11)}$	-1.06×10^4	0.40
$\ell^{(12)}$	-5.97×10^3	0.52
$\ell^{(13)}$	-1.01×10^4	0.39
$\ell^{(14)}$	-9.28×10^3	0.44
$\ell^{(15)}$	-7.09×10^3	0.57
$\ell^{(16)}$	-7.31×10^3	0.49
$\ell^{(17)}$	-5.82×10^3	0.53
$\ell^{(18)}$	-6.01×10^3	0.50
$\ell^{(19)}$	-2.19×10^4	0.23
$\ell^{(20)}$	-2.21×10^4	0.21
$\ell^{(21)}$	-2.01×10^4	0.23
$\ell^{(22)}$	-1.99×10^4	0.26
$\ell^{(23)}$	-2.03×10^4	0.26
$\ell^{(24)}$	-2.04×10^4	0.19

M.7 CUTS-GPR timings

Table M5: Timings for hyperparameter optimization and calculation of the predictive mean in CUTS-GPR. The speedup factor ($t_{\text{CPU}}/t_{\text{wall}}$) is 30–34 for hyperparameter optimization and 34–35 for predictions (using 36 cores for each calculation).

Molecule	Optimization		Predictive mean	
	CPU (h)	Wall (h)	CPU (min)	Wall (min)
Butadiene	63.6	1.95	64.8	1.83
DMSO	68.6	2.15	55.3	1.60
Ethylamine	59.6	1.94	60.9	1.73
Ethylene glycol	73.7	2.25	56.4	1.65
Nitroethane	64.2	1.91	65.8	1.88
Propanal	72.2	2.16	56.3	1.60
Pyrazine	65.6	1.96	65.2	1.84
Pyrrole	67.7	2.07	64.2	1.85
Thioacetone	109.1	3.58	55.0	1.58
Vinylformamide	50.1	1.62	57.8	1.63
Mean	69.4	2.16	60.2	1.72

M.8 SVGP timings

Table M6: Wall time for hyperparameter optimization and predictions with SVGP. The calculations were run on 36 CPU cores. The third and fourth columns indicate the interaction order, ω , and the number of inducing points, m .

Molecule	Method	ω	m	Optimization (h)	Predictions (min)
Butadiene	SVGP	2	256	2.36	1.98
			512	0.90	1.06
			1024	2.23	1.69
			2048	2.73	2.77
	SVGP	3	256	10.91	1.44
			512	8.76	2.00
			1024	16.34	4.01
DMSO	SVGP	2	256	1.50	0.84
			512	1.03	1.05
			1024	1.15	1.39
			2048	1.74	2.78
	SVGP	3	256	8.73	1.51
			512	7.36	2.27
			1024	10.91	3.23
Ethylamine	SVGP	2	256	1.41	0.96
			512	0.88	1.07
			1024	1.23	1.65
			2048	2.97	2.64
	SVGP	3	256	8.87	1.45
			512	8.71	2.35
			1024	13.80	3.38
Ethylene glycol	SVGP	2	256	1.88	0.83
			512	1.19	1.07
			1024	1.44	1.40
			2048	2.57	2.76
	SVGP	3	256	8.71	1.45
			512	8.29	2.37
			1024	15.39	6.03
Nitroethane	SVGP	2	256	1.30	0.84
			512	0.80	1.07
			1024	1.23	1.40
			2048	2.01	2.75
	SVGP	3	256	5.60	1.45
			512	9.37	2.32
			1024	10.31	3.80

Table M7: Wall time for hyperparameter optimization and predictions with SVGP. The calculations were run on 36 CPU cores. The third and fourth columns indicate the interaction order, ω , and the number of inducing points, m .

Molecule	Method	ω	m	Optimization (h)	Predictions (min)
Propanal	SVGP	2	256	1.49	0.84
			512	0.82	1.03
			1024	1.15	1.55
			2048	2.85	2.72
	SVGP	3	256	9.60	1.19
			512	10.07	2.34
			1024	16.98	4.72
Pyrazine	SVGP	2	256	0.85	0.83
			512	0.68	0.96
			1024	1.01	1.53
			2048	2.02	2.78
	SVGP	3	256	6.56	1.24
			512	8.43	2.32
			1024	8.33	4.35
Pyrrole	SVGP	2	256	1.65	0.95
			512	0.83	1.28
			1024	1.04	1.64
			2048	2.15	2.76
	SVGP	3	256	5.68	1.46
			512	8.01	2.36
			1024	14.50	3.98
Thioacetone	SVGP	2	256	0.90	1.08
			512	0.63	1.06
			1024	1.19	1.82
			2048	2.05	2.71
	SVGP	3	256	5.26	1.92
			512	4.98	2.36
			1024	9.35	4.89
Vinylformamide	SVGP	2	256	0.84	0.75
			512	0.88	1.08
			1024	1.25	1.60
			2048	2.57	2.78
	SVGP	3	256	8.33	1.35
			512	8.23	2.31
			1024	10.53	4.44

M.9 Comparison of average errors

Table M8: Comparison of range normalized prediction errors (dimensionless). The second and third columns indicate the interaction order, ω , and the number of inducing points, m , which is only applicable for the SVGP models. For each model, the average over the ten molecules in Table M1 is shown (with the standard deviation in parentheses).

Method	ω	m	MAX	MAE / 10^{-3}	RMSE / 10^{-3}
SVGP	2	256	0.698 (0.102)	7.55 (3.23)	15.84 (8.04)
		512	0.631 (0.123)	6.68 (2.79)	13.26 (6.24)
		1024	0.554 (0.088)	5.03 (2.21)	9.69 (4.00)
		2048	0.533 (0.076)	4.13 (1.76)	8.08 (3.19)
SVGP	3	256	0.605 (0.137)	5.80 (2.37)	10.83 (4.26)
		512	0.590 (0.112)	4.90 (2.09)	9.39 (3.49)
		1024	0.523 (0.108)	3.85 (1.93)	8.02 (3.32)
CUTS-GPR	3	—	0.168 (0.105)	1.58 (0.84)	4.31 (2.26)

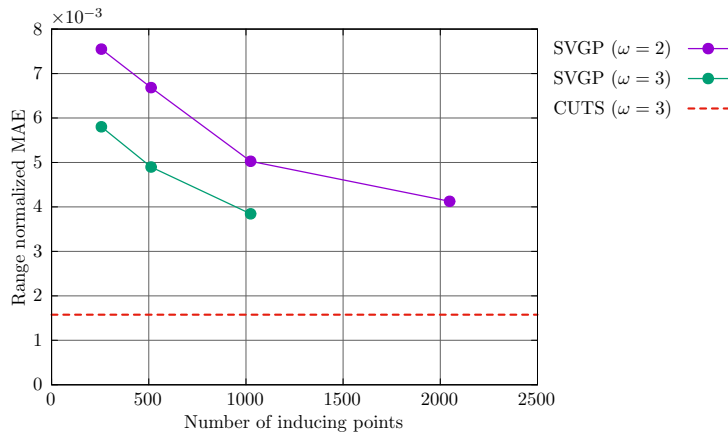


Figure M1: Range normalized MAE as a function of the number of inducing points. The errors are averaged over the ten molecules in Table M1.

M.10 Errors for all models and molecules

Table M9: Range normalized prediction errors (dimensionless).

Molecule	Method	ω	m	MAX	MAE	RMSE
Butadiene	SVGP	2	256	0.835	5.18×10^{-3}	1.28×10^{-2}
			512	0.747	4.49×10^{-3}	8.54×10^{-3}
			1024	0.665	3.87×10^{-3}	7.39×10^{-3}
			2048	0.594	3.12×10^{-3}	6.16×10^{-3}
	SVGP	3	256	0.679	4.74×10^{-3}	8.42×10^{-3}
			512	0.632	3.61×10^{-3}	7.29×10^{-3}
			1024	0.498	2.76×10^{-3}	5.63×10^{-3}
	CUTS-GPR	3	—	0.123	9.88×10^{-4}	2.91×10^{-3}
DMSO	SVGP	2	256	0.660	4.80×10^{-3}	8.24×10^{-3}
			512	0.630	4.71×10^{-3}	8.04×10^{-3}
			1024	0.627	3.68×10^{-3}	6.53×10^{-3}
			2048	0.550	3.43×10^{-3}	6.13×10^{-3}
	SVGP	3	256	0.756	3.90×10^{-3}	7.36×10^{-3}
			512	0.717	3.08×10^{-3}	6.17×10^{-3}
			1024	0.699	2.52×10^{-3}	5.46×10^{-3}
	CUTS-GPR	3	—	0.120	7.45×10^{-4}	1.76×10^{-3}
Ethylamine	SVGP	2	256	0.728	7.55×10^{-3}	1.85×10^{-2}
			512	0.423	6.69×10^{-3}	1.21×10^{-2}
			1024	0.507	5.18×10^{-3}	1.00×10^{-2}
			2048	0.521	3.92×10^{-3}	8.24×10^{-3}
	SVGP	3	256	0.471	6.59×10^{-3}	1.18×10^{-2}
			512	0.492	5.28×10^{-3}	1.03×10^{-2}
			1024	0.421	3.79×10^{-3}	8.72×10^{-3}
	CUTS-GPR	3	—	0.129	1.51×10^{-3}	4.18×10^{-3}
Ethylene glycol	SVGP	2	256	0.475	1.25×10^{-2}	2.22×10^{-2}
			512	0.493	1.26×10^{-2}	2.18×10^{-2}
			1024	0.534	9.91×10^{-3}	1.79×10^{-2}
			2048	0.542	8.46×10^{-3}	1.60×10^{-2}
	SVGP	3	256	0.390	1.10×10^{-2}	1.98×10^{-2}
			512	0.499	9.53×10^{-3}	1.66×10^{-2}
			1024	0.429	8.39×10^{-3}	1.52×10^{-2}
	CUTS-GPR	3	—	0.151	3.76×10^{-3}	1.02×10^{-2}
Nitroethane	SVGP	2	256	0.593	6.26×10^{-3}	1.01×10^{-2}
			512	0.498	5.02×10^{-3}	8.86×10^{-3}
			1024	0.401	4.39×10^{-3}	8.49×10^{-3}
			2048	0.446	3.64×10^{-3}	6.93×10^{-3}
	SVGP	3	256	0.496	4.72×10^{-3}	9.13×10^{-3}
			512	0.501	3.90×10^{-3}	7.79×10^{-3}
			1024	0.448	3.13×10^{-3}	6.72×10^{-3}
	CUTS-GPR	3	—	0.139	1.49×10^{-3}	3.80×10^{-3}

Table M10: Range normalized prediction errors (dimensionless).

Molecule	Method	ω	m	MAX	MAE	RMSE
Propanal	SVGP	2	256	0.754	8.18×10^{-3}	1.88×10^{-2}
			512	0.775	7.09×10^{-3}	1.57×10^{-2}
			1024	0.452	5.33×10^{-3}	1.04×10^{-2}
			2048	0.400	4.30×10^{-3}	8.60×10^{-3}
	SVGP	3	256	0.479	6.23×10^{-3}	1.18×10^{-2}
			512	0.409	5.52×10^{-3}	1.05×10^{-2}
			1024	0.380	4.10×10^{-3}	8.61×10^{-3}
	CUTS-GPR	3	—	0.115	1.81×10^{-3}	4.61×10^{-3}
Pyrazine	SVGP	2	256	0.692	7.55×10^{-3}	1.39×10^{-2}
			512	0.685	4.70×10^{-3}	9.73×10^{-3}
			1024	0.587	3.32×10^{-3}	7.10×10^{-3}
			2048	0.597	2.64×10^{-3}	5.91×10^{-3}
	SVGP	3	256	0.703	4.00×10^{-3}	7.84×10^{-3}
			512	0.675	3.52×10^{-3}	7.37×10^{-3}
			1024	0.614	2.42×10^{-3}	5.96×10^{-3}
	CUTS-GPR	3	—	0.162	1.03×10^{-3}	3.00×10^{-3}
Pyrrole	SVGP	2	256	0.758	4.56×10^{-3}	9.15×10^{-3}
			512	0.690	5.26×10^{-3}	1.05×10^{-2}
			1024	0.648	2.77×10^{-3}	5.85×10^{-3}
			2048	0.632	2.58×10^{-3}	5.37×10^{-3}
	SVGP	3	256	0.783	3.49×10^{-3}	6.87×10^{-3}
			512	0.726	2.95×10^{-3}	6.09×10^{-3}
			1024	0.608	2.27×10^{-3}	5.19×10^{-3}
	CUTS-GPR	3	—	0.136	1.10×10^{-3}	3.51×10^{-3}
Thioacetone	SVGP	2	256	0.759	1.39×10^{-2}	3.45×10^{-2}
			512	0.753	1.07×10^{-2}	2.65×10^{-2}
			1024	0.614	7.78×10^{-3}	1.55×10^{-2}
			2048	0.586	5.68×10^{-3}	1.06×10^{-2}
	SVGP	3	256	0.688	8.54×10^{-3}	1.65×10^{-2}
			512	0.686	7.23×10^{-3}	1.39×10^{-2}
			1024	0.629	5.92×10^{-3}	1.23×10^{-2}
	CUTS-GPR	3	—	0.464	1.72×10^{-3}	4.72×10^{-3}
Vinylformamide	SVGP	2	256	0.727	5.10×10^{-3}	1.03×10^{-2}
			512	0.620	5.57×10^{-3}	1.08×10^{-2}
			1024	0.502	4.04×10^{-3}	7.74×10^{-3}
			2048	0.458	3.51×10^{-3}	6.87×10^{-3}
	SVGP	3	256	0.601	4.84×10^{-3}	8.80×10^{-3}
			512	0.563	4.37×10^{-3}	7.78×10^{-3}
			1024	0.509	3.15×10^{-3}	6.40×10^{-3}
	CUTS-GPR	3	—	0.140	1.62×10^{-3}	4.39×10^{-3}